



MÁSTER EN CIBERSEGURIDAD

TRABAJO FIN DE MÁSTER

Auditoría de seguridad de Dolibarr ERP & CRM 17.0.0

TFM elaborado por: Jaime Hereza Niño

Tutor de TFM: Jonay Arón Puente Luis

Madrid, agosto de 2026

Resumen

El presente Trabajo Fin de Máster consiste en una auditoría de seguridad completa sobre la aplicación de gestión empresarial Dolibarr ERP & CRM en su versión 17.0.0, realizada mediante técnicas de hacking ético en un entorno de laboratorio local y aislado. El objetivo del trabajo es aplicar de forma integrada una metodología profesional de auditoría, basada en los estándares de OWASP, para identificar, explotar y documentar las vulnerabilidades presentes en esta versión, así como verificar los controles de seguridad que la aplicación implementa correctamente.

El trabajo recorre el ciclo completo de una auditoría de seguridad: preparación del entorno, reconocimiento, escaneo, explotación y análisis de resultados, culminando en la elaboración de un informe de auditoría con el formato que se entregaría a un cliente real. Durante la auditoría se han confirmado diez vulnerabilidades, tres de ellas de severidad crítica, incluyendo dos vectores de ejecución remota de código y un cross-site scripting almacenado a través de la API REST. Adicionalmente, se han verificado varios controles de seguridad que resisten los intentos de explotación, lo que permite un análisis contrastado del estado de seguridad de la aplicación.

Los resultados evidencian que Dolibarr 17.0.0 presenta debilidades críticas que comprometen por completo la confidencialidad, integridad y disponibilidad del sistema, y que la práctica totalidad de ellas se corrige en versiones posteriores, lo que subraya la importancia de una gestión de actualizaciones adecuada.

Palabras clave: hacking ético, auditoría de seguridad, pentesting, Dolibarr, OWASP, ASVS, vulnerabilidades web, ejecución remota de código, cross-site scripting, CVSS.

Abstract

This Master's Thesis presents a comprehensive security audit of the Dolibarr ERP & CRM business management application, version 17.0.0, carried out using ethical hacking techniques in an isolated local laboratory environment. The aim of the work is to apply an integrated, professional audit methodology, based on OWASP standards, in order to identify, exploit and document the vulnerabilities present in this version, as well as to verify the security controls that the application implements correctly.

The work covers the full life cycle of a security audit: environment preparation, reconnaissance, scanning, exploitation and analysis of results, culminating in the production of an audit report in the format that would be delivered to a real client. During the audit, ten vulnerabilities were confirmed, three of them of critical severity, including two remote code execution vectors and a stored cross-site scripting via the REST API. In addition, several security controls that withstand exploitation attempts were verified, allowing a contrasted analysis of the application's security posture.

The results show that Dolibarr 17.0.0 presents critical weaknesses that fully compromise the confidentiality, integrity and availability of the system, and that almost all of them are fixed in later versions, which highlights the importance of proper update management.

Keywords: ethical hacking, security audit, penetration testing, Dolibarr, OWASP, ASVS, web vulnerabilities, remote code execution, cross-site scripting, CVSS.

Índice

Resumen	2
Abstract	2
Índice	3
1. Introducción y objetivos	5
1.1. Contexto y motivación	5
1.2. Planteamiento del problema	5
1.3. Objetivos	5
Objetivo 1. Aplicar una metodología profesional de auditoría	5
Objetivo 2. Identificar, explotar y verificar las vulnerabilidades	6
Objetivo 3. Elaborar un informe de auditoría de nivel profesional	6
1.4. Estructura de la memoria	6
2. Marco teórico	7
2.1. Hacking ético y auditoría de seguridad	7
2.2. Metodologías y estándares de referencia	7
2.2.1. OWASP Application Security Verification Standard (ASVS 5.0)	7
2.2.2. OWASP Top 10	7
2.3. Clasificación y tipificación de vulnerabilidades	7
2.3.1. CVSS (Common Vulnerability Scoring System)	7
2.3.2. CWE y CVE	8
2.4. La aplicación objetivo: Dolibarr ERP & CRM	8
3. Preparación del entorno de laboratorio	9
3.1. Arquitectura del laboratorio	9
3.2. Despliegue de la aplicación objetivo	9
3.3. Configuración del escenario de auditoría	10
4. Reconocimiento y escaneo	10
4.1. Reconocimiento de la superficie	10
4.2. Análisis de cabeceras y configuración	10
4.3. Análisis del cifrado en tránsito	10
4.4. Enumeración de componentes	10
5. Explotación	11
5.1. Inyección y ejecución de código	11
5.1.1. Ejecución remota de código en el módulo Sitios Web	11
5.1.2. Ejecución remota de código mediante campos calculados	11
5.1.3. Cross-Site Scripting almacenado vía API REST	11
5.1.4. Verificación de inyección SQL (control correcto)	11
5.2. Control de acceso	11
5.2.1. Verificación de referencias directas a objetos (control correcto)	11
5.3. Gestión de ficheros	12
5.3.1. Verificación de subida de ficheros y path traversal (controles correctos)	12

5.4. Configuración y despliegue	12
5.4.1. Credenciales de base de datos en texto plano	12
5.4.2. Ausencia de cifrado en tránsito y otras deficiencias	12
5.5. Protección anti-CSRF (control correcto)	12
5.6. Cadena de ataque completa.....	12
6. Análisis y discusión de resultados.....	13
6.1. Resumen de resultados.....	13
6.2. Interpretación de los hallazgos	14
6.3. Discusión sobre la corrección en versiones posteriores	14
7. Conclusiones y trabajo futuro.....	15
7.1. Conclusiones.....	15
7.2. Valoración personal y competencias adquiridas	15
7.3. Líneas de trabajo futuro.....	15
Bibliografía.....	17
Anexo A. Informe de auditoría.....	18
Control de documentación	20
Índice	21
1. Resumen ejecutivo	23
1.1 Alcance y limitaciones	23
1.2 Entorno técnico	23
1.3 Planificación	23
1.4 Metodología.....	23
1.5 Resultados	24
2. Observaciones de la revisión	24
01. Ejecución remota de código en el módulo Sitios Web	25
02. Almacenamiento de credenciales en texto plano	28
03. Cross-Site Scripting almacenado a través de la API REST	30
04. Ejecución remota de código a través de campos calculados.....	32
05. Configuración deficiente de cifrado	34
06. Política de contraseñas deficiente.....	36
07. Configuración deficiente de cabeceras	39
08. Configuración deficiente en cookies.....	41
09. Piezas de software desactualizado	43
10. Divulgación de información sensible en mensajes de error	45
3. Conclusiones	48
4. Criterios de clasificación	49
5. Bitácora de verificación OWASP ASVS 5.0.....	50

1. Introducción y objetivos

1.1. Contexto y motivación

La seguridad de las aplicaciones web se ha convertido en una preocupación central para cualquier organización que gestione información sensible a través de sistemas conectados a Internet. Las soluciones de gestión empresarial de tipo ERP (Enterprise Resource Planning) y CRM (Customer Relationship Management) concentran datos especialmente valiosos (información de clientes, facturas, proveedores, empleados) y, por ello, constituyen un objetivo prioritario para los atacantes. Dentro de este panorama, el software libre ocupa un lugar destacado por su amplia adopción en pequeñas y medianas empresas, que frecuentemente carecen de los recursos necesarios para mantener sus sistemas debidamente actualizados y auditados.

La elección de este Trabajo Fin de Máster responde a una motivación personal clara: la seguridad ofensiva es, dentro de todo el campo de la ciberseguridad, el área que más me atrae desde que comencé mi formación. El hacking ético combina el rigor metodológico con una componente de resolución de problemas y creatividad técnica que encuentro especialmente estimulante. Esta inclinación me ha llevado a invertir una cantidad significativa de tiempo personal en formarme en la materia, obteniendo la certificación eJPT por el INE y completando numerosos laboratorios en plataformas como Hack The Box y TryHackMe, donde he practicado técnicas de explotación, escalada de privilegios y post-explotación en entornos controlados.

Este trabajo representa la oportunidad de aplicar de forma integrada, sobre un caso realista y con una metodología profesional, los conocimientos adquiridos tanto en el máster como en mi formación autodidacta, produciendo un resultado tangible y equivalente al de un encargo real de auditoría.

1.2. Planteamiento del problema

El problema que aborda este trabajo puede formularse de la siguiente manera: ¿qué nivel de seguridad presenta una versión concreta y no actualizada de una aplicación ERP/CRM ampliamente utilizada, y qué impacto tendría su explotación sobre una organización que la mantuviera en producción? Para responder a esta pregunta se ha seleccionado Dolibarr ERP & CRM 17.0.0, una versión que, por no estar actualizada, contiene vulnerabilidades conocidas que permiten ejercitar el ciclo completo de una auditoría de seguridad.

El interés del caso no reside únicamente en confirmar la existencia de vulnerabilidades conocidas, sino en verificarlas de forma empírica y defendible, analizar su impacto real mediante su explotación controlada, y contrastar los puntos débiles de la aplicación con aquellos controles que sí funcionan correctamente. Este enfoque contrastado aporta una visión más completa y realista del estado de seguridad de la aplicación que un simple listado de fallos.

1.3. Objetivos

El objetivo general de este trabajo es realizar una auditoría de seguridad profesional y completa sobre Dolibarr ERP & CRM 17.0.0, aplicando una metodología reconocida y documentando los resultados con el rigor exigible a un informe entregable a un cliente. Este objetivo general se concreta en los siguientes objetivos específicos:

Objetivo 1. Aplicar una metodología profesional de auditoría

Ejecutar el ciclo completo de una auditoría de seguridad web (preparación, reconocimiento, escaneo, explotación y análisis) siguiendo las metodologías de referencia del sector, en particular la OWASP Web Security Testing Guide (WSTG) para la ejecución de las pruebas y el OWASP

Application Security Verification Standard (ASVS 5.0) para la verificación estructurada de controles. Este objetivo persigue demostrar el dominio de un marco de trabajo sistemático y reproducible, frente a un enfoque no estructurado.

Objetivo 2. Identificar, explotar y verificar las vulnerabilidades

Detectar las vulnerabilidades presentes en la versión auditada y confirmarlas de forma empírica mediante su explotación controlada en el laboratorio, obteniendo evidencias defendibles de cada hallazgo. Este objetivo incluye la verificación de la aplicabilidad real de cada vulnerabilidad a la versión concreta, descartando aquellas que, pese a figurar en listados públicos, se encuentran ya corregidas, y documentando igualmente los controles que resisten los intentos de explotación.

Objetivo 3. Elaborar un informe de auditoría de nivel profesional

Producir un informe de resultados con la estructura, el rigor y la presentación equivalentes a los de un encargo profesional, que incluya un resumen ejecutivo, la clasificación de las vulnerabilidades según su severidad, la descripción de cada hallazgo con su impacto y recomendación, y las evidencias de explotación. Este objetivo busca demostrar la competencia no solo técnica, sino también de comunicación y documentación, esencial en el ejercicio profesional del pentesting.

1.4. Estructura de la memoria

La presente memoria se organiza en los siguientes capítulos. El capítulo 2 desarrolla el marco teórico, presentando los conceptos, metodologías y estándares en los que se apoya el trabajo. El capítulo 3 describe la preparación del entorno de laboratorio. El capítulo 4 detalla las fases de reconocimiento y escaneo. El capítulo 5 aborda la fase de explotación, que constituye el núcleo técnico del trabajo. El capítulo 6 presenta el análisis y la discusión de los resultados. El capítulo 7 recoge las conclusiones y las líneas de trabajo futuro. Finalmente, se incluyen la bibliografía y el anexo con el informe de auditoría completo.

2. Marco teórico

Este capítulo presenta los fundamentos conceptuales y metodológicos sobre los que se sustenta la auditoría. Se abordan sucesivamente el concepto de hacking ético y auditoría de seguridad, las metodologías y estándares empleados, los sistemas de clasificación y tipificación de vulnerabilidades, y una descripción de la aplicación objetivo.

2.1. Hacking ético y auditoría de seguridad

El hacking ético consiste en la aplicación de las mismas técnicas y herramientas que emplearía un atacante malicioso, pero de forma autorizada, controlada y con el objetivo de mejorar la seguridad del sistema evaluado. A diferencia de un ataque real, una auditoría de seguridad se desarrolla dentro de unos límites acordados (el alcance) y persigue identificar debilidades antes de que puedan ser explotadas con fines dañinos, proporcionando a la organización las recomendaciones necesarias para corregirlas.

Las auditorías de seguridad suelen clasificarse según el nivel de información del que dispone el auditor. En una auditoría de caja negra, el auditor no dispone de información previa ni de credenciales, simulando a un atacante externo. En una auditoría de caja blanca, dispone de acceso completo al código fuente, la configuración y la documentación. La auditoría de caja gris, empleada en este trabajo, se sitúa en un punto intermedio: el auditor dispone de credenciales de usuario con privilegios limitados, además de la superficie accesible sin autenticar, lo que permite evaluar tanto la exposición externa como los controles internos de la aplicación.

2.2. Metodologías y estándares de referencia

Para dotar de rigor y reproducibilidad a la auditoría se han empleado varias metodologías y estándares reconocidos en el sector, principalmente promovidos por OWASP (Open Worldwide Application Security Project), organización de referencia en seguridad de aplicaciones web.

2.2.1. OWASP Application Security Verification Standard (ASVS 5.0)

El OWASP ASVS es un estándar de verificación que define un conjunto estructurado de requisitos de seguridad, organizados en capítulos y niveles de exigencia. El estándar establece tres niveles: el Nivel 1, verificable mediante técnicas de caja negra y caja gris, se considera el mínimo exigible para cualquier aplicación; los niveles 2 y 3 requieren acceso a la documentación, el código fuente y las personas implicadas en el desarrollo. En este trabajo se ha verificado el Nivel 1, cuyo resultado detallado se recoge en la bitácora de pruebas incluida en el anexo.

2.2.2. OWASP Top 10

El OWASP Top 10 es un documento de concienciación que recopila las diez categorías de riesgo más críticas en aplicaciones web. En este trabajo se emplea como marco de comunicación, permitiendo relacionar cada hallazgo con una categoría de riesgo ampliamente reconocida y facilitando así la comprensión del impacto por parte de audiencias no especializadas.

2.3. Clasificación y tipificación de vulnerabilidades

2.3.1. CVSS (Common Vulnerability Scoring System)

El CVSS es el estándar de facto para la valoración de la severidad de las vulnerabilidades. En su versión 3.1, empleada en este trabajo, calcula una puntuación numérica de 0 a 10 a partir de un vector de métricas que consideran el vector de ataque, la complejidad, los privilegios requeridos, la interacción del usuario y el impacto sobre la confidencialidad, integridad y disponibilidad. Esta puntuación permite priorizar objetivamente la corrección de las vulnerabilidades.

2.3.2. CWE y CVE

El CWE (Common Weakness Enumeration) es un catálogo de tipos de debilidades de software, que permite clasificar la naturaleza técnica de cada vulnerabilidad (por ejemplo, CWE-79 para cross-site scripting). El CVE (Common Vulnerabilities and Exposures) es un sistema de identificación única de vulnerabilidades concretas y públicamente conocidas (por ejemplo, CVE-2023-30253). Ambos se emplean en este trabajo para tipificar y referenciar cada hallazgo de forma inequívoca.

2.4. La aplicación objetivo: Dolibarr ERP & CRM

Dolibarr ERP & CRM es una aplicación web de gestión empresarial de código abierto, distribuida bajo licencia GPLv3 y desarrollada en PHP. Su modularidad y facilidad de despliegue la han convertido en una solución popular entre pequeñas y medianas empresas, autónomos y asociaciones, que la emplean para gestionar clientes, facturas, productos, proyectos y otros aspectos de su actividad.

La versión seleccionada para este trabajo es la 17.0.0, publicada como parte de la serie 17 de la aplicación. Su elección se justifica por tratarse de una versión no actualizada que contiene vulnerabilidades conocidas y públicamente documentadas, lo que la convierte en un objetivo idóneo para ejercitar una auditoría completa. Es importante destacar que las vulnerabilidades explotadas en este trabajo se corrigen mayoritariamente en la versión 17.0.1, aspecto que se retoma en la discusión de resultados.

Conviene señalar que la imagen oficial de Dolibarr no publica un contenedor específico para la versión 17.0.0 exacta, por lo que el laboratorio se ha construido a partir del código fuente de dicha versión, como se detalla en el capítulo 3.

3. Preparación del entorno de laboratorio

Este capítulo describe el diseño y la construcción del entorno de laboratorio sobre el que se ha realizado la auditoría. La correcta preparación del entorno es una fase crítica que garantiza que las pruebas se realizan de forma controlada, aislada y reproducible, sin afectar a sistemas de terceros ni a la red de producción.

3.1. Arquitectura del laboratorio

El laboratorio se ha desplegado íntegramente sobre una única máquina con sistema operativo Kali Linux, que actúa simultáneamente como máquina de auditoría y como anfitrión del entorno objetivo. La aplicación objetivo se ejecuta mediante contenedores Docker y se publica exclusivamente en la interfaz de loopback, de modo que no resulta accesible desde la red local.

El aislamiento del entorno se sustenta en dos elementos: por un lado, la publicación del servicio únicamente en la dirección de loopback impide el acceso desde otros equipos de la red; por otro, la red bridge de Docker separa el entorno de contenedores del resto del sistema.

Máquina de auditoría	Kali Linux
Dirección de red	192.168.0.38
Objetivo publicado en	127.0.0.1:80
Motor de contenedores	Docker con red bridge
Herramienta de interceptación	Burp Suite

3.2. Despliegue de la aplicación objetivo

Dado que la imagen oficial de Dolibarr no ofrece un contenedor para la versión 17.0.0 exacta, el entorno se ha construido a partir del código fuente de dicha versión, desplegado sobre una imagen base de PHP con Apache. La orquestación de los servicios (el servidor web con la aplicación y el servidor de base de datos MariaDB) se ha realizado mediante Docker Compose, definiendo una red interna que comunica ambos contenedores.

El proceso de despliegue requirió la instalación manual de varias extensiones de PHP exigidas por el instalador de Dolibarr y por algunos de sus módulos, así como la configuración de los permisos necesarios sobre los ficheros de configuración y el directorio de documentos.

Aplicación	Dolibarr ERP & CRM 17.0.0 (GPLv3)
Servidor web	Apache/2.4.65
Intérprete	PHP 8.1.34
Base de datos	MariaDB 10.6
Imagen base	php:8.1-apache

3.3. Configuración del escenario de auditoría

Para reproducir un escenario realista equivalente al de una pyme en producción, la instancia se configuró activando los módulos de negocio habituales (Terceros, Usuarios y grupos, API REST y Sitios Web) en lugar de mantener la instalación mínima por defecto.

Asimismo, se crearon cuentas de usuario con distintos niveles de privilegio, incluyendo una cuenta de administrador y una cuenta de usuario con privilegios limitados. Esta última resulta imprescindible para evaluar correctamente los controles de acceso.

4. Reconocimiento y escaneo

Este capítulo recoge las fases de reconocimiento y escaneo, orientadas a obtener información sobre la aplicación objetivo y a identificar su superficie de ataque antes de proceder a la explotación. Estas fases permiten mapear los puntos de entrada, detectar tecnologías y versiones, y localizar posibles debilidades de configuración.

4.1. Reconocimiento de la superficie

El reconocimiento inicial se centró en identificar los recursos accesibles de la aplicación y las tecnologías empleadas. Mediante herramientas de descubrimiento de rutas se localizaron los principales puntos de entrada.

4.2. Análisis de cabeceras y configuración

La inspección de las cabeceras de respuesta HTTP permitió identificar tanto controles correctamente implementados como deficiencias de configuración. Se comprobó la presencia de cabeceras de seguridad correctas, y a su vez, la ausencia de cabeceras recomendadas por OWASP así como cabeceras mal configuradas.

4.3. Análisis del cifrado en tránsito

El análisis de la configuración criptográfica del servicio, realizado con la herramienta testssl, confirmó que la aplicación se servía exclusivamente a través de HTTP, sin ninguna capa de cifrado TLS.

4.4. Enumeración de componentes

La identificación de los componentes de terceros empleados por la aplicación reveló el uso de librerías front-end en versiones desactualizadas. La detección de versiones se realizó combinando el análisis del cliente web con la inspección directa de los recursos servidos, verificando así la información obtenida por medios automáticos.

5. Explotación

Este capítulo constituye el núcleo técnico del trabajo. En él se detallan las vulnerabilidades explotadas durante la auditoría, organizadas por categorías, así como los controles de seguridad que, tras ser sometidos a prueba, demostraron funcionar correctamente.

Cada vulnerabilidad se ha confirmado de forma empírica mediante su explotación controlada, obteniendo evidencias defendibles de su existencia.

5.1. Inyección y ejecución de código

5.1.1. Ejecución remota de código en el módulo Sitios Web

El hallazgo más grave de la auditoría se localiza en el módulo «Sitios Web», que permite crear páginas web dinámicas con código PHP. La aplicación implementa un filtro para bloquear la etiqueta de apertura de PHP, pero dicho filtro es sensible a mayúsculas y minúsculas. Empleando una variante de la etiqueta con la capitalización alterada se evade por completo la comprobación, ya que el intérprete de PHP la procesa con independencia de su forma. Al no existir en este punto una lista negra de funciones, se logra la ejecución de comandos arbitrarios del sistema operativo con los privilegios del servidor web.

La explotación se confirmó introduciendo un payload en el editor de la página que, al ser renderizada por su URL pública, ejecutó un comando del sistema cuya salida identificó al usuario del servidor web.

5.1.2. Ejecución remota de código mediante campos calculados

Un segundo vector de ejecución de código se localiza en la funcionalidad de campos calculados, cuyo contenido se evalúa dinámicamente como código PHP. En este caso, la aplicación sí implementa una lista negra de funciones peligrosas, pero este enfoque resulta insuficiente: se demostró la ejecución de código empleando una función no incluida en la lista, que devolvió la versión del intérprete de PHP.

5.1.3. Cross-Site Scripting almacenado vía API REST

La API REST de la aplicación permite crear y modificar objetos de negocio. Se comprobó que el filtro de saneamiento de entradas no cubre determinados vectores de inyección basados en atributos de eventos HTML.

Un payload introducido a través de la API se almacena sin sanear y se ejecuta al visualizar el objeto en la interfaz web, materializando un cross-site scripting almacenado con origen en la API.

5.1.4. Verificación de inyección SQL (control correcto)

Se sometieron a prueba los vectores clásicos de inyección SQL sobre los parámetros de ordenación, los filtros de búsqueda y los identificadores numéricos. En todos los casos la aplicación validó la entrada contra listas blancas de campos, trató los payloads como literales o forzó el tipo entero, mostrando que la capa de acceso a base de datos parametriza correctamente las consultas.

5.2. Control de acceso

5.2.1. Verificación de referencias directas a objetos (control correcto)

Para evaluar la existencia de referencias directas a objetos inseguras (IDOR), se empleó una cuenta de usuario con privilegios limitados, con permiso de consulta de terceros pero sin la ampliación de acceso a todos los objetos. Al intentar acceder por identificador directo a objetos pertenecientes a otro usuario, tanto por URL como a través del listado, la aplicación denegó el acceso.

5.3. Gestión de ficheros

5.3.1. Verificación de subida de ficheros y path traversal (controles correctos)

Se intentó subir ficheros con extensiones ejecutables a través de la funcionalidad de documentos. En todos los casos la aplicación neutralizó los ficheros añadiendo automáticamente un sufijo que impide su interpretación por el servidor, incluyendo variantes con distintas extensiones y capitalizaciones. De forma análoga, los intentos de explotar el script de descarga de documentos mediante secuencias de path traversal fueron bloqueados por el saneamiento de rutas y por el filtro global de la aplicación. Ambos controles funcionan correctamente.

5.4. Configuración y despliegue

5.4.1. Credenciales de base de datos en texto plano

Se verificó que el fichero de configuración principal almacena la contraseña de la base de datos en texto plano, al encontrarse desactivada la opción de cifrado que la aplicación ofrece. Esta debilidad, combinada con las vulnerabilidades de ejecución de código, permite que un atacante que obtenga ejecución de comandos lea directamente las credenciales y acceda a la totalidad de la base de datos, constituyendo un eslabón crítico de la cadena de compromiso.

5.4.2. Ausencia de cifrado en tránsito y otras deficiencias

Además de los hallazgos anteriores, se confirmaron la ausencia de cifrado en tránsito, la política de contraseñas insuficiente, las deficiencias en las cabeceras de seguridad, la configuración inadecuada de las cookies, el uso de componentes desactualizados y la divulgación de información en los mensajes de error. Todos estos hallazgos se documentan de forma detallada, con su impacto y recomendación, en el informe de auditoría anexo.

5.5. Protección anti-CSRF (control correcto)

Se comprobó la protección frente a ataques de falsificación de petición en sitios cruzados (CSRF) intentando reenviar una petición de creación de usuario sin el token correspondiente. La aplicación rechazó la petición, confirmando que la protección basada en tokens funciona correctamente. Este control solo podría eludirse robando previamente el token mediante un cross-site scripting.

5.6. Cadena de ataque completa

Las vulnerabilidades identificadas cobran su máxima dimensión al combinarse en una cadena de compromiso. La secuencia demostrada durante la auditoría parte de una cuenta de usuario con privilegios bajos, favorecida por la política de contraseñas insuficiente, que, a través de la ejecución remota de código del módulo Sitios Web, obtiene ejecución de comandos como usuario del servidor web. Desde esa posición, el atacante lee el fichero de configuración y obtiene las credenciales de la base de datos en texto plano, alcanzando finalmente el control total sobre la información de negocio almacenada.

Esta cadena ilustra cómo una vulnerabilidad de severidad media, como la política de contraseñas, actúa como facilitador de un compromiso crítico, y por qué la defensa en profundidad y la actualización del software constituyen medidas prioritarias. El detalle completo de cada eslabón se recoge en el informe de auditoría anexo.

6. Análisis y discusión de resultados

6.1. Resumen de resultados

La auditoría ha permitido confirmar diez vulnerabilidades sobre la instancia de Dolibarr 17.0.0, distribuidas según su nivel de criticidad conforme al estándar CVSS v3.1. La siguiente figura resume dicha distribución:



Figura 1: Distribución de vulnerabilidades por nivel de criticidad.

Junto a las vulnerabilidades confirmadas, la auditoría ha verificado cinco controles de seguridad que resisten los intentos de explotación: la protección anti-CSRF, la neutralización de ficheros ejecutables, el control de acceso a objetos, la protección frente a path traversal y la parametrización frente a inyección SQL. La siguiente tabla sintetiza ambos conjuntos de resultados:

Ref.	Vulnerabilidad	Criticidad	Estado
RSW-0826026001-01	Ejecución remota de código en el módulo Sitios Web	Crítica	Pendiente
RSW-0826026001-02	Almacenamiento de credenciales en texto plano	Crítica	Pendiente
RSW-0826026001-03	Cross-Site Scripting almacenado a través de la API REST	Crítica	Pendiente
RSW-0826026001-04	Ejecución remota de código a través de campos calculados	Alta	Pendiente
RSW-0826026001-05	Configuración deficiente de cifrado	Media	Pendiente
RSW-0826026001-06	Política de contraseñas deficiente	Media	Pendiente
RSW-0826026001-07	Configuración deficiente de cabeceras	Baja	Pendiente
RSW-0826026001-08	Configuración deficiente en cookies	Baja	Pendiente
RSW-0826026001-09	Piezas de software desactualizado	Baja	Pendiente
RSW-0826026001-10	Divulgación de información sensible en mensajes de error	Baja	Pendiente

6.2. Interpretación de los hallazgos

Los resultados obtenidos permiten extraer una lectura que va más allá del simple recuento de vulnerabilidades. El aspecto más significativo es el marcado contraste entre las distintas áreas de seguridad de la aplicación. Por un lado, los controles relacionados con la validación de entradas y la ejecución de código presentan fallos graves, que permiten comprometer por completo el sistema. Por otro, los controles de acceso a ficheros y a objetos (IDOR, path traversal, subida de ficheros) resisten adecuadamente los intentos de explotación.

6.3. Discusión sobre la corrección en versiones posteriores

Un elemento de discusión relevante es que la práctica totalidad de las vulnerabilidades críticas y altas identificadas se corrigen en la versión 17.0.1, publicada poco después de la 17.0.0. Este hecho refuerza una de las conclusiones centrales del trabajo: buena parte del riesgo al que se expone una organización no deriva de la inexistencia de correcciones, sino de la falta de aplicación de las actualizaciones disponibles.

7. Conclusiones y trabajo futuro

7.1. Conclusiones

La realización de este Trabajo Fin de Máster ha permitido alcanzar los objetivos planteados en el capítulo introductorio. Respecto al primer objetivo, se ha aplicado con éxito una metodología profesional de auditoría, ejecutando el ciclo completo (preparación, reconocimiento, escaneo, explotación y análisis) con el apoyo de los estándares de OWASP, lo que ha aportado sistematicidad y rigor al proceso.

En cuanto al segundo objetivo, se han identificado, explotado y verificado las vulnerabilidades presentes en la versión auditada, confirmando de forma empírica diez hallazgos y documentando adicionalmente cinco controles que funcionan correctamente. La conclusión técnica más relevante es contundente: Dolibarr 17.0.0 es una versión críticamente vulnerable, que presenta dos vectores independientes de ejecución remota de código y permite, mediante su encadenamiento con otras debilidades, el compromiso total del sistema. Ninguna organización debería mantener esta versión en producción sin actualizar.

Respecto al tercer objetivo, se ha elaborado un informe de auditoría con el formato, el rigor y la presentación equivalentes a los de un encargo profesional, que se incluye como anexo de esta memoria. Con ello se demuestra la competencia no solo en la vertiente técnica de la auditoría, sino también en la comunicación y documentación de sus resultados, aspecto esencial en el ejercicio profesional.

En una valoración global, el resultado del trabajo es satisfactorio, este trabajo representa, para mí, un ejemplo más de auditoría completa sobre el que consolidar la metodología y las competencias adquiridas, un entorno adicional analizado que se suma a la práctica continuada que considero imprescindible en este campo.

7.2. Valoración personal y competencias adquiridas

A nivel personal, este trabajo ha supuesto la oportunidad de integrar en un caso realista los conocimientos adquiridos a lo largo del máster con la formación autodidacta en la que he invertido una parte importante de mi tiempo, incluyendo la obtención de la certificación eJPT y la resolución de numerosos laboratorios en plataformas como Hack The Box y TryHackMe. La seguridad ofensiva es el área que más me atrae de la ciberseguridad, y este trabajo ha reforzado tanto mis competencias técnicas como mi convicción de orientar mi carrera profesional hacia el pentesting.

De cara a mi desarrollo futuro, me propongo continuar mi especialización a través de certificaciones progresivamente más exigentes, comenzando por la eWPT y la certificación oficial de Burp Suite en el ámbito de la seguridad web, y avanzando hacia la OSCP como siguiente hito relevante, antes de abordar certificaciones de nivel superior.

Considero que el ejercicio continuado y la práctica diaria en este sector es obligatorio para mantenerse al día.

7.3. Líneas de trabajo futuro

A partir del trabajo realizado, se identifican varias líneas de continuación que permitirían ampliar y profundizar en sus resultados:

- Comparación entre versiones: auditar de forma comparada la versión 17.0.0 y la 17.0.1 para analizar en detalle las correcciones aplicadas, evaluando la calidad de cada parche, por ejemplo, si la corrección de los filtros basados en listas negras es completa o simplemente amplía la lista.

- Ampliación del alcance: extender la auditoría a otros módulos de Dolibarr no cubiertos en este trabajo, así como a otras aplicaciones ERP/CRM de código abierto, con el fin de contrastar sus posturas de seguridad.
- Profundización en la explotación: desarrollar cadenas de post-explotación completas a partir de los vectores de ejecución de código confirmados, evaluando el alcance real de un compromiso en un escenario de producción.
- Automatización: elaborar herramientas o scripts propios que automaticen parte de las comprobaciones realizadas, agilizando la verificación de estos hallazgos en futuras auditorías.

Bibliografía

- Dolibarr Foundation. (2023). Dolibarr ERP & CRM - Documentation. Recuperado de <https://www.dolibarr.org/>
- FIRST. (2019). Common Vulnerability Scoring System version 3.1: Specification Document. Forum of Incident Response and Security Teams. Recuperado de <https://www.first.org/cvss/>
- MITRE. (s. f.). Common Weakness Enumeration (CWE). Recuperado de <https://cwe.mitre.org/>
- MITRE. (s. f.). Common Vulnerabilities and Exposures (CVE). Recuperado de <https://cve.mitre.org/>
- NIST. (s. f.). National Vulnerability Database (NVD). National Institute of Standards and Technology. Recuperado de <https://nvd.nist.gov/>
- OWASP Foundation. (2021). OWASP Top 10 - 2021. Recuperado de <https://owasp.org/Top10/>
- OWASP Foundation. (s. f.). OWASP Web Security Testing Guide (WSTG). Recuperado de <https://owasp.org/www-project-web-security-testing-guide/>
- OWASP Foundation. (2024). OWASP Application Security Verification Standard (ASVS). Recuperado de <https://owasp.org/www-project-application-security-verification-standard/>
- PortSwigger. (s. f.). Web Security Academy. Recuperado de <https://portswigger.net/web-security>

Anexo A. Informe de auditoría

El presente anexo contiene el informe de auditoría de seguridad completo, elaborado con el formato y el rigor de un entregable profesional, que constituye uno de los principales resultados de este TFM. En él se detalla cada vulnerabilidad con su clasificación, descripción, impacto, recomendación y evidencias de explotación, los criterios de clasificación empleados y la bitácora completa de verificación de controles OWASP ASVS 5.0.



MÁSTER EN CIBERSEGURIDAD

TRABAJO FIN DE MÁSTER

Auditoría de seguridad de Dolibarr ERP & CRM 17.0.0

TFM elaborado por: Jaime Hereza Niño
Tutor de TFM: Jonay Arón Puente Luis

Madrid, agosto de 2026

Control de documentación

Referencia del informe	RSW-0826026001
Versión	1.0
Tipo de documento	Informe de resultados - Auditoría de seguridad
Fecha	Agosto de 2026
Autor	Jaime Hereza Niño
Tutor / Revisión	Jonay Arón Puente Luis
Aplicación auditada	Dolibarr ERP & CRM v17.0.0
Metodologías	OWASP · OWASP ASVS 5.0

Aviso de confidencialidad

El presente documento tiene carácter estrictamente académico y ha sido elaborado como Trabajo Fin de Máster. La auditoría descrita se ha realizado sobre un entorno de laboratorio local y aislado, propiedad del autor, sin afectar en ningún momento a sistemas de terceros.

Índice

Resumen	2
Abstract	2
Índice	3
1. Introducción y objetivos	5
1.1. Contexto y motivación	5
1.2. Planteamiento del problema	5
1.3. Objetivos	5
Objetivo 1. Aplicar una metodología profesional de auditoría	5
Objetivo 2. Identificar, explotar y verificar las vulnerabilidades	6
Objetivo 3. Elaborar un informe de auditoría de nivel profesional	6
1.4. Estructura de la memoria	6
2. Marco teórico	7
2.1. Hacking ético y auditoría de seguridad	7
2.2. Metodologías y estándares de referencia	7
2.2.1. OWASP Application Security Verification Standard (ASVS 5.0)	7
2.2.2. OWASP Top 10	7
2.3. Clasificación y tipificación de vulnerabilidades	7
2.3.1. CVSS (Common Vulnerability Scoring System)	7
2.3.2. CWE y CVE	8
2.4. La aplicación objetivo: Dolibarr ERP & CRM	8
3. Preparación del entorno de laboratorio	9
3.1. Arquitectura del laboratorio	9
3.2. Despliegue de la aplicación objetivo	9
3.3. Configuración del escenario de auditoría	10
4. Reconocimiento y escaneo	10
4.1. Reconocimiento de la superficie	10
4.2. Análisis de cabeceras y configuración	10
4.3. Análisis del cifrado en tránsito	10
4.4. Enumeración de componentes	10
5. Explotación	11
5.1. Inyección y ejecución de código	11
5.1.1. Ejecución remota de código en el módulo Sitios Web	11
5.1.2. Ejecución remota de código mediante campos calculados	11
5.1.3. Cross-Site Scripting almacenado vía API REST	11
5.1.4. Verificación de inyección SQL (control correcto)	11
5.2. Control de acceso	11
5.2.1. Verificación de referencias directas a objetos (control correcto)	11
5.3. Gestión de ficheros	12
5.3.1. Verificación de subida de ficheros y path traversal (controles correctos)	12

5.4. Configuración y despliegue	12
5.4.1. Credenciales de base de datos en texto plano	12
5.4.2. Ausencia de cifrado en tránsito y otras deficiencias	12
5.5. Protección anti-CSRF (control correcto)	12
5.6. Cadena de ataque completa.....	12
6. Análisis y discusión de resultados.....	13
6.1. Resumen de resultados.....	13
6.2. Interpretación de los hallazgos	14
6.3. Discusión sobre la corrección en versiones posteriores	14
7. Conclusiones y trabajo futuro.....	15
7.1. Conclusiones.....	15
7.2. Valoración personal y competencias adquiridas	15
7.3. Líneas de trabajo futuro.....	15
Bibliografía.....	17
Anexo A. Informe de auditoría.....	18
Control de documentación	20
Índice	21
1. Resumen ejecutivo	23
1.1 Alcance y limitaciones	23
1.2 Entorno técnico	23
1.3 Planificación	23
1.4 Metodología.....	23
1.5 Resultados	24
2. Observaciones de la revisión	24
01. Ejecución remota de código en el módulo Sitios Web	25
02. Almacenamiento de credenciales en texto plano	28
03. Cross-Site Scripting almacenado a través de la API REST	30
04. Ejecución remota de código a través de campos calculados.....	32
05. Configuración deficiente de cifrado	34
06. Política de contraseñas deficiente.....	36
07. Configuración deficiente de cabeceras	39
08. Configuración deficiente en cookies.....	41
09. Piezas de software desactualizado	43
10. Divulgación de información sensible en mensajes de error	45
3. Conclusiones	48
4. Criterios de clasificación	49
5. Bitácora de verificación OWASP ASVS 5.0.....	50

1. Resumen ejecutivo

El presente informe recoge los resultados de la auditoría de seguridad realizada sobre la aplicación Dolibarr ERP & CRM en su versión 17.0.0.

El objetivo del trabajo ha sido identificar y documentar las vulnerabilidades presentes en esta versión mediante técnicas de hacking ético, siguiendo una metodología profesional equivalente a la de un encargo real de pentesting, en el marco de un entorno de laboratorio controlado.

1.1 Alcance y limitaciones

El entorno auditado ha sido una única instancia de Dolibarr ERP & CRM 17.0.0, desplegada mediante contenedores Docker en un entorno de laboratorio local y aislado. El objetivo se ha publicado exclusivamente en la interfaz de loopback, sin exposición a la red:

- Aplicación: Dolibarr ERP & CRM 17.0.0.
- Dirección del objetivo: `http://127.0.0.1:80`.
- Máquina de auditoría Kali Linux: 192.168.0.38.

La instancia se configuró con los módulos de negocio habituales de una pyme (Terceros, Usuarios, API REST y Sitios Web) para reproducir un escenario realista de uso.

Se ha adoptado un enfoque de auditoría de caja gris: se dispone de credenciales de usuario con privilegios limitados..

1.2 Entorno técnico

La instancia auditada se ha desplegado sobre la siguiente pila tecnológica, montada a partir del código fuente de Dolibarr 17.0.0:

Aplicación	Dolibarr ERP & CRM 17.0.0
Servidor web	Apache/2.4.65
Intérprete	PHP 8.1.34
Base de datos	MariaDB 10.6
Contenedores	Docker
Publicación	127.0.0.1:80

1.3 Planificación

La auditoría se ha desarrollado cubriendo las fases de preparación del laboratorio, reconocimiento, escaneo, explotación, análisis de resultados y redacción del informe.

1.4 Metodología

La auditoría se ha apoyado en las metodologías y estándares de referencia de OWASP:

- OWASP ASVS 5.0 (Application Security Verification Standard) para la verificación estructurada de controles. Se ha comprobado el Nivel 1, que es completamente revisable mediante técnicas de pentesting de caja negra y caja gris, y se considera el mínimo exigible para cualquier aplicación.
- OWASP Top 10 como marco de referencia para la comunicación de las categorías de riesgo identificadas.

1.5 Resultados

Durante la auditoría se han identificado 10 vulnerabilidades, distribuidas según su nivel de criticidad de la siguiente manera:



Figura 1: Distribución de vulnerabilidades por nivel de criticidad.

1.6 Tabla resumen de hallazgos

Ref.	Vulnerabilidad	Criticidad	Estado
RSW-0826026001-01	Ejecución remota de código en el módulo Sitios Web	Crítica	Pendiente
RSW-0826026001-02	Almacenamiento de credenciales en texto plano	Crítica	Pendiente
RSW-0826026001-03	Cross-Site Scripting almacenado a través de la API REST	Crítica	Pendiente
RSW-0826026001-04	Ejecución remota de código a través de campos calculados	Alta	Pendiente
RSW-0826026001-05	Configuración deficiente de cifrado	Media	Pendiente
RSW-0826026001-06	Política de contraseñas deficiente	Media	Pendiente
RSW-0826026001-07	Configuración deficiente de cabeceras	Baja	Pendiente
RSW-0826026001-08	Configuración deficiente en cookies	Baja	Pendiente
RSW-0826026001-09	Piezas de software desactualizado	Baja	Pendiente
RSW-0826026001-10	Divulgación de información sensible en mensajes de error	Baja	Pendiente

2. Observaciones de la revisión

Esta sección detalla cada una de las vulnerabilidades identificadas durante la auditoría. Para cada hallazgo se incluye una tabla de cabecera con sus metadatos y clasificación, seguida de su descripción, el impacto derivado, la recomendación de mitigación y las evidencias de explotación.

01. Ejecución remota de código en el módulo Sitios Web

RSW-0826026001-01	Crítica
Entorno afectado :	Dolibarr ERP/CRM 17.0.0
Funcionalidad :	Módulo Sitios Web
Estado :	Pendiente
Tipo de vulnerabilidad :	Validación
CVSS v3.1 :	8.8

Descripción

El módulo «Sitios Web» de Dolibarr permite a los usuarios crear y editar páginas web dinámicas que pueden incluir código PHP. Para evitar la ejecución de código arbitrario introducido por el usuario, la aplicación implementa un filtro que bloquea la etiqueta de apertura de PHP dentro del contenido del editor de páginas. Esta vulnerabilidad se corresponde con la referencia CVE-2023-30253.

Durante la auditoría se ha comprobado que dicho filtro es sensible a mayúsculas y minúsculas. Empleando una variante de la etiqueta de apertura con la capitalización alterada se evade por completo la comprobación, ya que el intérprete de PHP procesa la etiqueta con independencia de su forma. La página maliciosa se almacena en el servidor y su código se ejecuta al solicitar la URL pública de renderizado.

Durante la revisión se han observado los siguientes aspectos:

- El filtro bloquea la cadena literal «<?php» pero no contempla variantes como «<?pHp», que el motor de PHP interpreta igualmente.
- A diferencia de otros puntos de la aplicación, este editor no aplica lista negra de funciones, por lo que funciones peligrosas como system() se ejecutan sin restricción.
- La ejecución se produce con los privilegios del usuario del servidor web (www-data), confirmada mediante la salida del comando id.

Impacto derivado

La explotación de esta vulnerabilidad permite a un usuario autenticado con privilegios bajos ejecutar comandos arbitrarios del sistema operativo sobre el servidor que aloja la aplicación. Esto compromete de forma total la confidencialidad, integridad y disponibilidad del sistema, ya que un atacante puede leer, modificar o eliminar cualquier información accesible por el servicio web, así como interrumpir su funcionamiento.

A partir de la ejecución de comandos, un atacante podría leer ficheros sensibles del servidor (incluido el fichero de configuración con las credenciales de la base de datos), pivotar hacia otros sistemas de la red interna, establecer mecanismos de persistencia o desplegar una webshell completa. Dado que el vector solo requiere privilegios bajos, cualquier cuenta de usuario válida resulta suficiente para comprometer el servidor por completo.

Recomendación

Se recomienda actualizar Dolibarr a una versión igual o superior a la 17.0.1, en la que el filtro de este módulo fue corregido. Como medida complementaria, debe sanearse la entrada del editor mediante una comprobación insensible a mayúsculas y minúsculas que contemple todas las variantes de las etiquetas de apertura de PHP, en lugar de una comparación literal que resulta trivial de evadir.

Explotación

A continuación se presentan las evidencias que acreditan la existencia de la vulnerabilidad:

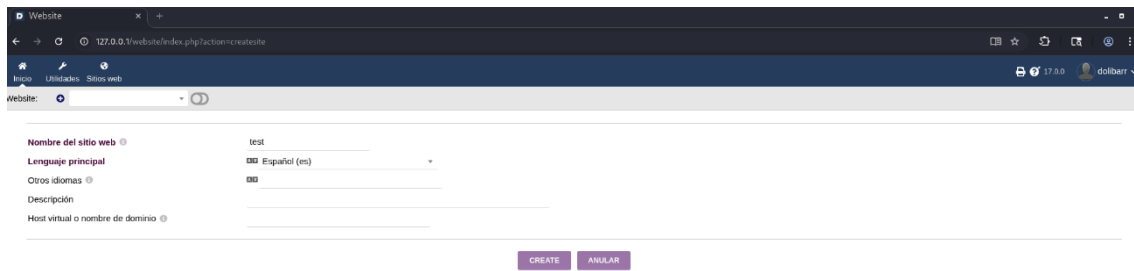


Figura 1: Configuración de nueva web

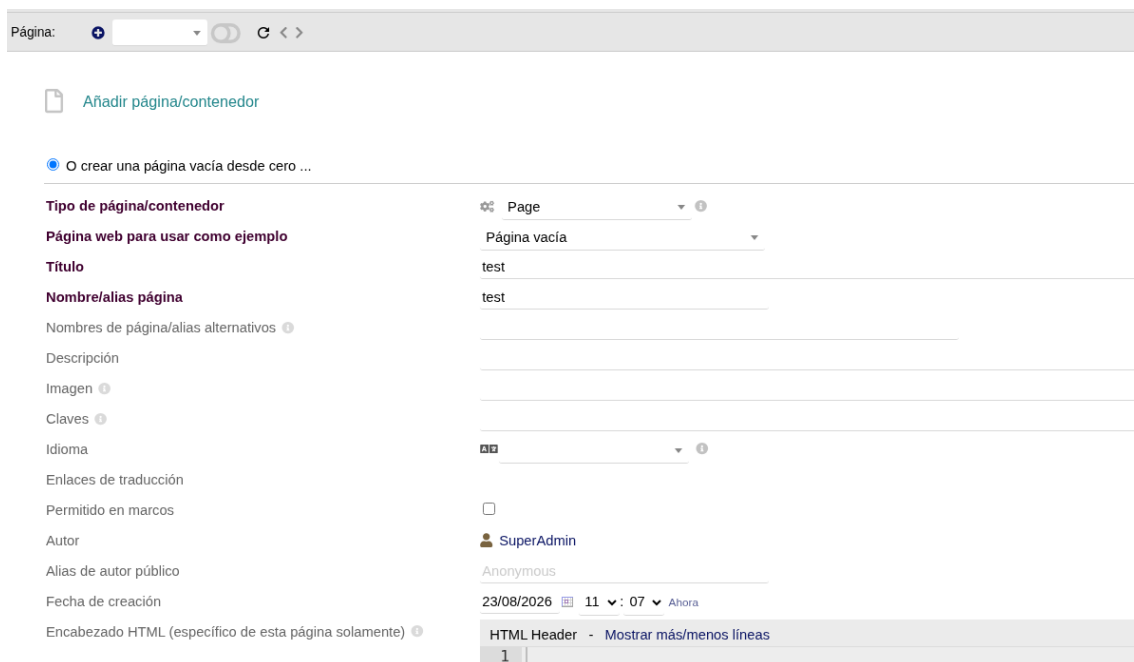


Figura 2: Configuración de la página

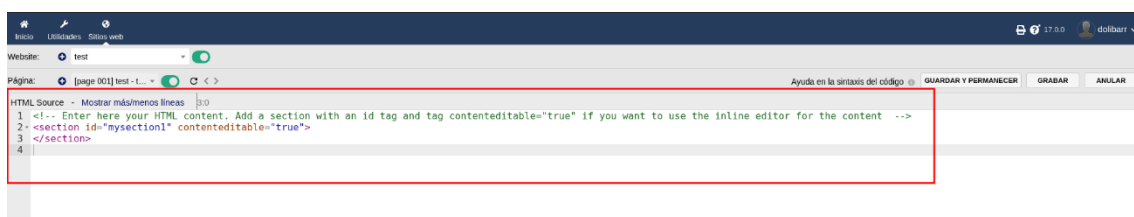


Figura 3: HTML sin payload

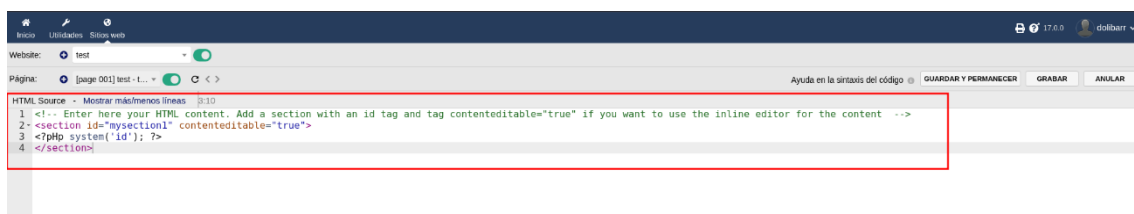


Figura 4: Introducción del payload



Figura 5: Confirmación de la vulnerabilidad RCE

02. Almacenamiento de credenciales en texto plano

RSW-0826026001-02	Crítica
Entorno afectado :	Dolibarr ERP/CRM 17.0.0
Funcionalidad :	Fichero de configuración conf.php
Estado :	Pendiente
Tipo de vulnerabilidad :	Configuración
CVSS v3.1 :	9.1

Descripción

El fichero de configuración principal de Dolibarr almacena la contraseña de acceso a la base de datos MariaDB en texto plano. Aunque la aplicación ofrece una opción de configuración para cifrar este valor, dicha opción se encuentra desactivada por defecto en la instalación estándar.

Durante la auditoría se ha verificado el contenido del fichero, confirmando que la directiva que almacena la contraseña de la base de datos contiene el valor sin ningún tipo de cifrado ni ofuscación. Esta debilidad cobra especial relevancia al combinarse con las vulnerabilidades de ejecución de código identificadas en este mismo informe.

Durante la revisión se han observado los siguientes aspectos:

- La opción «Cifrar la contraseña de la base de datos almacenada en conf.php» está disponible pero desactivada.
- Combinada con una ejecución remota de código, permite el acceso directo y completo a la base de datos.

Impacto derivado

La exposición de las credenciales de la base de datos en texto plano permite a cualquier atacante que consiga acceso de lectura al sistema de ficheros del servidor obtener acceso total a la base de datos. Esto compromete la confidencialidad e integridad de toda la información de negocio almacenada, incluyendo datos de clientes, facturas y las credenciales cifradas de los usuarios de la aplicación.

El acceso directo a la base de datos permitiría a un atacante extraer información sensible, manipular registros o eliminarlos por completo, con un impacto severo sobre el conjunto de la organización. Esta vulnerabilidad constituye un eslabón crítico de la cadena de compromiso, ya que convierte una ejecución de comandos limitada en un control total de los datos del sistema.

Recomendación

Se recomienda activar la opción de cifrado de la contraseña de la base de datos disponible en el apartado de configuración de seguridad de Dolibarr. Esta medida evita que la credencial quede expuesta en claro ante un acceso no autorizado al sistema de ficheros, elevando de forma sustancial la protección del componente más sensible de la configuración de la aplicación.

Deben restringirse los permisos del fichero de configuración al mínimo imprescindible para el usuario del servidor web, y establecerse credenciales de base de datos robustas, distintas de las de otros servicios y sometidas a una política de rotación periódica. Estas prácticas reducen tanto la probabilidad de exposición como el impacto de un eventual compromiso.

Explotación

A continuación se presentan las evidencias que acreditan la existencia de la vulnerabilidad:



```
(jaime@ jaime)-[redacted]  
$ docker exec -it $(docker-compose ps -q web) grep -i "dolibarr_main_db_pass" /var/www/html/conf/conf.php  
$dolibarr_main_db_pass='dolibarr';
```

Figura 6: Almacenamiento de credenciales en texto plano

03. Cross-Site Scripting almacenado a través de la API REST

RSW-0826026001-03	Crítica
Entorno afectado :	Dolibarr ERP/CRM 17.0.0
Funcionalidad :	API REST - Terceros
Estado :	Pendiente
Tipo de vulnerabilidad :	Validación
CVSS v3.1 :	9.6

Descripción

La API REST de Dolibarr permite crear y modificar objetos de negocio mediante peticiones autenticadas. Los datos introducidos por esta vía son procesados por una función de saneamiento encargada de filtrar el contenido potencialmente malicioso antes de su almacenamiento en la base de datos. Esta vulnerabilidad se corresponde con la referencia CVE-2023-38888.

Se ha comprobado que dicho filtro no cubre determinados vectores de inyección basados en atributos de eventos HTML. Un payload introducido en un campo de texto de un tercero a través de la API supera el filtro y se almacena sin sanear. Al visualizar la ficha en la interfaz web, el dato se renderiza sin codificación de salida, ejecutándose el código en el navegador de la víctima.

Durante la revisión se han observado los siguientes aspectos:

- El dato se almacena de forma persistente en la base de datos y ejecuta al renderizar la ficha del objeto.
- Al originarse en la API y ejecutar en la web, afecta a cualquier usuario que consulte la ficha, incluidos administradores.

Impacto derivado

La inyección de código JavaScript persistente permite a un atacante ejecutar acciones en el contexto del navegador de cualquier usuario legítimo que visualice el objeto afectado, incluidos los administradores. A diferencia de un XSS reflejado, no requiere engañar a la víctima para que siga un enlace: basta con que consulte una ficha aparentemente normal para que el código se ejecute.

El impacto potencial incluye el robo de sesiones de usuario, la ejecución de acciones en nombre de la víctima, como la creación de usuarios o la modificación de datos, y el robo del token anti-CSRF. Esto último es especialmente grave, ya que permite encadenar esta vulnerabilidad con otras que la protección anti-CSRF mitigaría por sí sola, ampliando notablemente la superficie de ataque efectiva.

Recomendación

Se recomienda reforzar el filtro de entrada para contemplar vectores basados en atributos de eventos HTML, como onerror u onload, además de la etiqueta <script>. La implementación de una política de seguridad de contenido (Content-Security-Policy) que limite la ejecución de scripts en línea añade una capa de defensa complementaria frente a este tipo de ataques.

Explotación

A continuación se presentan las evidencias que acreditan la existencia de la vulnerabilidad:

Usuario | Permisos 21 | Interfaz usuario | Nota | Documentos | Log

Título de cortesía

Apellidos SuperAdmin

Nombre

Login dolibarr

Administrador Sí

Sexo

Empleado ☒

Supervisor Nada

Usuario externo ? Interno

Intervalo de fechas de validez de acceso Ahora

Contraseña

Clave para API 9S2bDxp78na224qWIS8RHPnAf1pV7EKp

Figura 7: API

```
(jaime@jaime)-[redacted]
$ curl -s -X POST "http://127.0.0.1/api/index.php/thirdparties" \
-H "DOLAPIKEY: 9S2bDxp78na224qWIS8RHPnAf1pV7EKp" \
-H "Content-Type: application/json" \
-d '{"name": "PoC XSS API", "client": "1", "name_alias": "<img src=x onerror=alert(document.domain)>"}'
```

Figura 8: Petición a la API REST con el payload XSS

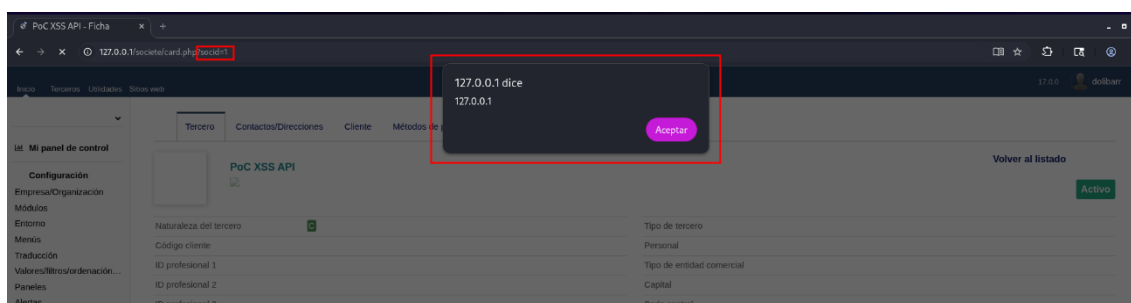


Figura 9: Confirmación de la vulnerabilidad XSS

04. Ejecución remota de código a través de campos calculados

RSW-0826026001-04	Alta
Entorno afectado :	Dolibarr ERP/CRM 17.0.0
Funcionalidad :	Campos adicionales
Estado :	Pendiente
Tipo de vulnerabilidad :	Validación
CVSS v3.1 :	7.2

Descripción

Dolibarr permite definir campos adicionales sobre distintos objetos de la aplicación. Entre las opciones de estos campos se encuentra el «valor calculado», cuyo contenido se evalúa dinámicamente mediante una función interna que interpreta y ejecuta código PHP en el contexto del servidor. Esta vulnerabilidad se corresponde con la referencia CVE-2024-40137.

Se ha comprobado que es posible introducir expresiones PHP en este campo, que son ejecutadas al visualizar la ficha del objeto correspondiente. La aplicación implementa una lista negra de funciones peligrosas, pero este enfoque resulta insuficiente, ya que cualquier función no contemplada en la lista permite eludir la protección y ejecutar código arbitrario.

Durante la revisión se han observado los siguientes aspectos:

- El campo «valor calculado» se procesa mediante `dol_eval()`, que interpreta código PHP.
- Se ha demostrado la ejecución con una función no filtrada (`phpversion()`), que devolvió la versión del intérprete (8.1.34).

Impacto derivado

La ejecución de código PHP arbitrario en el servidor compromete la confidencialidad, integridad y disponibilidad del sistema. Aunque la explotación requiere privilegios elevados sobre la configuración de campos adicionales, la existencia de una política de contraseñas débil en la misma instancia reduce la barrera de acceso efectiva para un atacante.

El enfoque de mitigación basado en lista negra es intrínsecamente frágil, ya que depende de enumerar todas las funciones peligrosas conocidas. Cualquier función omitida o alternativa permite eludir la protección, por lo que el riesgo de una evolución del payload hacia la ejecución de comandos del sistema operativo es elevado, con el consiguiente compromiso total del servidor.

Recomendación

Se recomienda sustituir el modelo de lista negra por uno de lista blanca, que permita únicamente un conjunto reducido y seguro de operaciones, o bien eliminar por completo la evaluación dinámica de código en campos definidos por el usuario.

También debe restringirse el permiso de gestión de campos adicionales a un número mínimo de administradores de confianza, aplicando el principio de mínimo privilegio. Reducir el número de cuentas capaces de definir estos campos disminuye la superficie de ataque y limita la exposición ante el compromiso de una cuenta con privilegios de configuración.

Explotación

A continuación se presentan las evidencias que acreditan la existencia de la vulnerabilidad:

Configuración del módulo usuarios Volver al listado de módulos

Parámetros Grupo Campos adicionales (Usuarios) 1 Campos adicionales (Grupos)

Define cualquier atributo adicional/personalizado que deba agregarse a: Usuarios

Posición	Clave de traducción o cadena	Cadena traducida	Código	Tipo	Tamaño	Campo combinado	Único	Obligatorio	Puede editarse siempre	Visibilidad	Mostrar en PDF	Totalizable	Css en páginas de edición	Css en las páginas de visualización	Css en páginas de lista
100	pwn	pwn	pwn	Texto (1 línea)	255	hexdec(1) ? sys...	No	No	Si	1	0	No			

Edición del campo pwn

Clave de traducción o cadena: pwn

Código: pwn

Tipo: Texto (1 línea)

Tamaño: 255

Posición: 100

Archivo de idioma

Campo combinado: phpversion()

Figura 10: Inserción del payload

SuperAdmin Activado

Volver al listado

Login	dolibarr	pwn	8.1.34
Tipo	Interno	Firma	
Empleado	☒	Cartas credenciales	
Supervisor	Nada	Intervalo de fechas de validez de acceso	
Puesto de trabajo		Última conexión	23/08/2026 12:32 (Anterior), 23/08/2026 12:33 (Actualmente)
Horas trabajadas (por semana)			
Salario			
Tasa media por hora			
Tasa media por día			
Fecha empleo			
Fecha de nacimiento			

Figura 11: Confirmación de la vulnerabilidad RCE

05. Configuración deficiente de cifrado

RSW-0826026001-05	Media
Entorno afectado :	Dolibarr ERP/CRM 17.0.0
Funcionalidad :	Servicio web
Estado :	Pendiente
Tipo de vulnerabilidad :	Cifrado
CVSS v3.1 :	5.9

Descripción

El servicio web de la aplicación se expone exclusivamente a través del protocolo HTTP, sin ninguna capa de cifrado TLS. La comprobación realizada con la herramienta testssl.sh confirma que no se ofrece ningún protocolo seguro y que no existe un certificado de servidor asociado al servicio.

En consecuencia, toda la información intercambiada entre el cliente y el servidor viaja en texto plano por la red, incluidas las credenciales de acceso y los tokens de sesión. Cualquier intermediario con acceso al tráfico de red puede interceptar y leer esta información sin necesidad de descifrado alguno.

Durante la revisión se han observado los siguientes aspectos:

- Todos los protocolos seguros aparecen como «not offered».
- No se dispone de certificado de servidor.
- Las credenciales de acceso se transmiten en claro durante el proceso de autenticación.

Impacto derivado

La ausencia de cifrado en tránsito expone las comunicaciones a ataques de interceptación de tipo man-in-the-middle. Un atacante con acceso a la red, por ejemplo en una red compartida o comprometida, podría capturar credenciales de acceso, tokens de sesión y datos de negocio directamente del tráfico, sin necesidad de vulnerar ningún otro control.

El compromiso de las credenciales o de los tokens de sesión permitiría a un atacante suplantar a usuarios legítimos y acceder a la aplicación con sus privilegios. Esto afecta tanto a la confidencialidad de la información transmitida como a la integridad de las sesiones de usuario, y facilita la explotación de otras vulnerabilidades que requieren acceso autenticado.

Recomendación

Se recomienda configurar el servidor web para servir la aplicación exclusivamente a través de HTTPS mediante un certificado TLS válido. Todo el tráfico HTTP debe redirigirse de forma permanente a HTTPS, y las cookies de sesión deben marcarse con el atributo Secure para evitar su transmisión por canales no cifrados.

Explotación

A continuación se presentan las evidencias que acreditan la existencia de la vulnerabilidad:

```
Testing protocols via sockets except NPN+ALPN

SSLv2      not offered (OK)
SSLv3      not offered (OK)
TLS 1      not offered
TLS 1.1    not offered
TLS 1.2    not offered
TLS 1.3    not offered

You should not proceed as no protocol was detected. If you still really really want to, say "YES" --> YES
NPN/SPDY   not offered
ALPN/HTTP2 not offered

Testing cipher categories

NULL ciphers (no encryption)           not offered (OK)
Anonymous NULL Ciphers (no authentication) not offered (OK)
Export ciphers (w/o ADH+NULL)          not offered (OK)
LOW: 64 Bit + DES, RC[2,4], MD5 (w/o export) not offered (OK)
Triple DES Ciphers / IDEA              not offered
Obsolete CBC ciphers (AES, ARIA etc.)   not offered
Strong encryption (AEAD ciphers) with no FS not offered
Forward Secrecy strong encryption (AEAD ciphers) not offered
```

Figura 12: Protocolos TLS no ofrecidos

```
Testing robust forward secrecy (FS) -- omitting Null Authentication/Encryption, 3DES, RC4

No ciphers supporting Forward Secrecy (FS) offered

Testing server defaults (Server Hello)

TLS extensions (standard)    (none)
Session Ticket RFC 5077 hint no -- no lifetime advertised
SSL Session ID support       no
Session Resumption           Tickets no, ID: no
TLS clock skew               SSLv3 through TLS 1.2 didn't return a timestamp
Client Authentication         none

Client problem: No server certificate could be retrieved. Thus we can't continue with "server defaults".
```

Figura 13: Ausencia de algoritmo de cifrado para Forward Secrecy

06. Política de contraseñas deficiente

RSW-0826026001-06	Media
Entorno afectado :	Dolibarr ERP/CRM 17.0.0
Funcionalidad :	Gestión de usuarios
Estado :	Pendiente
Tipo de vulnerabilidad :	Autenticación
CVSS v3.1 :	5.3

Descripción

La instancia auditada aplica el validador de contraseñas «Standard» de Dolibarr, que únicamente impone una longitud mínima de 12 caracteres. Este validador no exige complejidad alguna, es decir, no requiere combinar mayúsculas, minúsculas, números y símbolos en la composición de la contraseña.

Se ha comprobado que el sistema acepta contraseñas basadas en palabras de diccionario que cumplen la longitud mínima pero resultan triviales de adivinar mediante ataques automatizados. El validador con reglas de complejidad configurables se encuentra disponible en la aplicación, pero está desactivado en la configuración actual.

Durante la revisión se han observado los siguientes aspectos:

- El validador «Standard» valida únicamente la longitud mínima (12 caracteres), no la complejidad.
- Se aceptó la contraseña «contraseñadebil», compuesta solo por minúsculas y basada en diccionario.
- El validador «Perso», que permite exigir complejidad, se encuentra desactivado.

Impacto derivado

Una política que valida la longitud pero no la complejidad permite el uso de contraseñas predecibles, facilitando los ataques de diccionario y de fuerza bruta contra las cuentas de usuario. Dado que la aplicación no aplica bloqueo de cuenta tras intentos fallidos, un atacante puede realizar múltiples pruebas de autenticación sin restricciones significativas.

El compromiso de una cuenta, incluso de bajo privilegio, puede servir como punto de entrada para explotar otras vulnerabilidades identificadas en este informe, como la ejecución remota de código. De este modo, una debilidad de severidad media actúa como facilitador de un compromiso de mayor gravedad, elevando el riesgo global del despliegue.

Recomendación

Se recomienda activar y configurar el validador «Perso» de Dolibarr para exigir complejidad además de longitud, requiriendo la combinación de mayúsculas, minúsculas, números y caracteres especiales. Esta medida eleva de forma significativa la resistencia de las contraseñas frente a ataques de diccionario y fuerza bruta.

Conviene implementar un mecanismo de bloqueo temporal de cuentas tras un número determinado de intentos fallidos de autenticación, y considerar la comprobación de las contraseñas frente a listas de credenciales comprometidas conocidas. Estas prácticas refuerzan la seguridad del proceso de autenticación de forma complementaria a la política de complejidad.

Explotación

A continuación se presentan las evidencias que acreditan la existencia de la vulnerabilidad:

 Configuración de la seguridad

Elija el método que se utilizará para las contraseñas generadas automáticamente.

Miscelánea
 Contraseñas
 Archivos (Enviar archivo)
 Acceso externo/internet
 Eventos de seguridad
 Permisos por defecto

Reglas para generar y validar contraseñas.		Ejemplo	Activado
 None	No sugerir ninguna contraseña generada. La contraseña debe ser escrita manualmente. Longitud mínima: 0	Nada	☐
 Standard	Devuelve una contraseña generada según el algoritmo interno de Dolibarr: 12 caracteres que contienen números y caracteres compartidos. Longitud mínima: 12	8krvvhj7lad5	☒
 Perso	Devuelve una contraseña según la configuración definida. Longitud mínima: Obteniendo tu configuración	8st3rqQfNHM5	☐

Parámetros	Activado
Cifrar las contraseñas almacenadas en la base de datos (NO en texto plano). Se recomienda encarecidamente activar esta opción.	☒ Desactivar
Cifrar la contraseña de la base de datos almacenada en conf.php. Se recomienda encarecidamente activar esta opción.	☐ Activar
No mostrar el vínculo "Contraseña olvidada" en la página de login	☐ Activar

Figura 14: Política de contraseñas débil

 Nuevo usuario

Este formulario le permite crear un usuario interno para su empresa/organización. Para crear un usuario externo (cliente, proveedor del tercero).

Título de cortesía

▼ ⓘ

Apellidos
 prueba

Nombre
 prueba

Login
 usuario

Administrador
 No ▼

Sexo
 Hom... ▼

Empleado
☒

Supervisor



▼

Usuario externo ?
 Interno ⓘ

Intervalo de fechas de validez de acceso

📅

Ahora

📅

Contraseña
 contraseñadebil

↺

Clave para API

↺

Figura 15: Prueba crear usuario contraseña debil

 bitos web

Este usuario no tiene permisos definidos

Usuario

Permisos 0

Interfaz usuario

Nota

Documentos

Log

 prueba prueba  

Volver al listado < >

Activado

Login	usuario	pwn	8.1.34
Tipo	Interno	Enlace terceros / contactos	Este usuario no es un cliente potencial, cliente o proveedor
Empleado	☒	Firma	
Supervisor	Nada		
Puesto de trabajo			
Horas trabajadas (por semana)			
Fecha empleo			
Fecha de nacimiento			

Cartas credenciales

Intervalo de fechas de validez de acceso

Clave para API

Última conexión

ENVIAR E-MAIL

MODIFICAR

GENERAR NUEVA CONTRASEÑA

ENVIAR NUEVA CONTRASEÑA

DESACTIVAR

ELIMINAR

Figura 16: Usuario creado con éxito

07. Configuración deficiente de cabeceras

RSW-0826026001-07	Baja
Entorno afectado :	Dolibarr ERP/CRM 17.0.0
Funcionalidad :	Cabeceras de respuesta HTTP
Estado :	Pendiente
Tipo de vulnerabilidad :	Configuración
CVSS v3.1 :	3.7

Descripción

Durante la revisión de las cabeceras de respuesta HTTP del servidor se han identificado deficiencias en la configuración de las cabeceras de seguridad. Estas cabeceras contribuyen a la defensa en profundidad de la aplicación, instruyendo al navegador para que aplique restricciones adicionales que reducen la superficie de ataque del lado del cliente. A continuación se detallan las cabeceras ausentes recomendadas por OWASP y aquellas que se encuentran mal configuradas.

Cabeceras ausentes

- Content-Security-Policy: no se envía. Es la principal defensa frente a la inyección de scripts (XSS), al permitir restringir las fuentes desde las que el navegador puede cargar y ejecutar contenido.
- Strict-Transport-Security (HSTS): no se envía. Fuerza al navegador a comunicarse exclusivamente por HTTPS, evitando ataques de degradación de protocolo.
- Permissions-Policy: no se envía. Permite restringir el acceso a funcionalidades del navegador (cámara, micrófono, geolocalización, etc.) que la aplicación no necesita.

Cabeceras mal configuradas

- Cabeceras Server y X-Powered-By: exponen las versiones exactas del software del servidor (Apache y PHP), proporcionando información útil a un atacante durante la fase de reconocimiento. Deberían suprimirse o generalizarse.

Se ha verificado que sí están presentes y correctamente configuradas las cabeceras X-Content-Type-Options (nosniff), X-Frame-Options (SAMEORIGIN) y Referrer-Policy (same-origin), lo que indica una configuración parcial pero incompleta.

Impacto derivado

La ausencia de estas cabeceras reduce las capas de defensa que el navegador puede aplicar frente a ataques como la inyección de scripts, la degradación de la conexión a HTTP o el abuso de funcionalidades del navegador. Aunque por sí sola no constituye una vía de compromiso directa, debilita el conjunto de protecciones disponibles.

En particular, la falta de una política de seguridad de contenido incrementa el impacto potencial de otras vulnerabilidades, como el XSS almacenado identificado en este informe, al no existir una barrera adicional que limite la ejecución de scripts en línea. La combinación de ambas debilidades amplía las posibilidades de explotación de un atacante.

Recomendación

Se recomienda implementar las cabeceras ausentes que recomienda OWASP y configurar aquellas para que se consideren seguras.

Explotación

A continuación se presentan las evidencias que acreditan la existencia de la vulnerabilidad:



```
(jaime@jaime)-[~/Documentos/dolibarr]
$ curl -I http://127.0.0.1
HTTP/1.1 200 OK
Date: Sun, 23 Aug 2026 10:33:51 GMT
Server: Apache/2.4.65 (Debian)
X-Powered-By: PHP/8.1.34
Set-Cookie: DOLSESSID_c00a75cac6d3c98334d62ee0488b5f2312f30e2b=dbeea9a9f2ed9bd0f50b45dc126596ed; path=/; HttpOnly; SameSite=Lax
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: Public, must-revalidate
Pragma: no-cache
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: same-origin
Content-Type: text/html; charset=UTF-8
```

Figura 17: Ausencia y mala configuración de cabeceras

08. Configuración deficiente en cookies

RSW-0826026001-08	Baja
Entorno afectado :	Dolibarr ERP/CRM 17.0.0
Funcionalidad :	Gestión de sesiones
Estado :	Pendiente
Tipo de vulnerabilidad :	Sesión
CVSS v3.1 :	3.1

Descripción

Se ha comprobado que las cookies emitidas por la aplicación no incluyen todos los atributos de seguridad recomendados. Estos atributos determinan cómo el navegador gestiona, transmite y protege las cookies, y su ausencia debilita la protección de las sesiones de usuario frente a distintos vectores de ataque.

En concreto, la cookie de sesión se emite sin el atributo Secure, como consecuencia directa del uso de HTTP, y una cookie auxiliar carece del atributo HttpOnly y se configura con una política SameSite permisiva. Esta configuración expone las cookies a interceptación y a robo mediante scripts del lado del cliente.

Durante la revisión se han observado los siguientes aspectos:

- La cookie de sesión DOLSESSID se emite sin el atributo Secure.
- La cookie DOLINSTALLNOPING carece del atributo HttpOnly y usa SameSite=None.
- La falta de Secure es consecuencia directa de la ausencia de HTTPS en el servicio.

Impacto derivado

La ausencia del atributo Secure permite que la cookie de sesión se transmita por canales no cifrados, exponiéndola a interceptación por parte de un atacante con acceso a la red. Esto facilita el robo de la sesión y la suplantación del usuario legítimo sin necesidad de conocer sus credenciales de acceso.

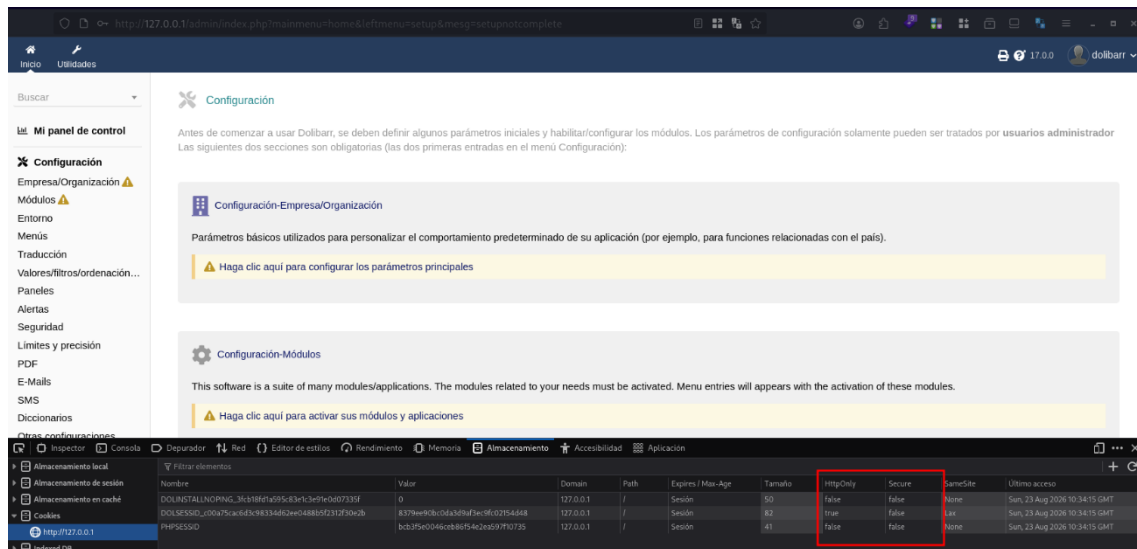
La ausencia del atributo HttpOnly facilita el robo de cookies mediante ataques de tipo XSS, al permitir que el código JavaScript inyectado acceda a su valor. Combinada con una política SameSite permisiva, esta configuración incrementa la superficie de ataque frente a vectores de tipo cross-site, ampliando las posibilidades de compromiso de la sesión.

Recomendación

Se recomienda marcar todas las cookies de sesión con los atributos Secure y HttpOnly una vez habilitado HTTPS. El atributo Secure garantiza que las cookies solo se transmitan por canales cifrados, mientras que HttpOnly impide su acceso desde el código JavaScript, mitigando el robo de sesión mediante XSS.

Explotación

A continuación se presentan las evidencias que acreditan la existencia de la vulnerabilidad:



The screenshot shows the Dolibarr configuration interface. The left sidebar contains a menu with options like 'Mi panel de control', 'Configuración', 'Empresa/Organización', 'Módulos', 'Entorno', 'Menús', 'Traducción', 'Valores/filtros/ordenación...', 'Pantallas', 'Alertas', 'Seguridad', 'Límites y precisión', 'PDF', 'E-Mails', 'SMS', and 'Diccionarios'. The main content area is titled 'Configuración' and contains two sections: 'Configuración-Empresa/Organización' and 'Configuración-Módulos'. Below these sections, a table lists cookies. The 'HttpOnly' and 'Secure' columns are highlighted with a red box, showing that these security features are disabled for several cookies.

Nombre	Valor	Domain	Path	Expires / Max-Age	Tamaño	HttpOnly	Secure	SameSite	Último acceso
DOLIBARR_INSTALLING_3f318f1d295434c3d5f0d0733f	0	127.0.0.1	/	Sesión	50	false	false	None	Sun, 23 Aug 2020 10:34:15 GMT
DOLIBARR_SESSION_0c678c3a6d4c4e33462e0488a9d231273092b	8379e9f8b3c0d240f3ec9f02164448	127.0.0.1	/	Sesión	82	true	false	Lax	Sun, 23 Aug 2020 10:34:15 GMT
PHPSESSID	bcb35e0046c88054c2e597f0735	127.0.0.1	/	Sesión	81	false	false	None	Sun, 23 Aug 2020 10:34:15 GMT

Figura 18: Configuración deficiente de cookies

09. Piezas de software desactualizado

RSW-0826026001-09	Baja
Entorno afectado :	Dolibarr ERP/CRM 17.0.0
Funcionalidad :	Librerías front-end
Estado :	Pendiente
Tipo de vulnerabilidad :	Mantenimiento
CVSS v3.1 :	3.1

Descripción

La aplicación incorpora librerías de terceros en versiones desactualizadas. El empleo de componentes desarrollados por terceros es una práctica habitual, pero requiere un mantenimiento continuo, ya que estas piezas pueden incluir vulnerabilidades conocidas que se corrigen en versiones posteriores.

Durante la revisión se han identificado las librerías jQuery y jQuery UI en versiones para las que existen actualizaciones posteriores con correcciones de seguridad. El uso de versiones sin actualizar introduce en la aplicación vulnerabilidades públicamente documentadas que un atacante podría aprovechar.

Durante la revisión se han observado los siguientes aspectos:

- Se ha detectado jQuery 3.6.0, con versiones posteriores disponibles.
- Se ha detectado jQuery UI 1.13.2, igualmente desactualizada.
- La falta de actualización es indicativa de un proceso de gestión de dependencias deficiente.

Impacto derivado

Las librerías desactualizadas pueden contener vulnerabilidades conocidas que un atacante podría aprovechar, especialmente en combinación con otras debilidades de la aplicación. Aunque el riesgo directo asociado a estas versiones concretas es bajo, su presencia amplía la superficie de ataque disponible.

Más allá del riesgo concreto de cada versión, la falta de mantenimiento de las dependencias es un indicador de una gestión de parches deficiente, que suele correlacionarse con la presencia de otras vulnerabilidades no corregidas. Un proceso de actualización inconsistente incrementa la probabilidad de que existan fallos conocidos sin mitigar.

Recomendación

Se recomienda actualizar las librerías jQuery y jQuery UI a sus versiones estables más recientes, incorporando así las correcciones de seguridad publicadas. Esta actualización debe realizarse verificando la compatibilidad con el resto de la aplicación para evitar regresiones en su funcionamiento.

Explotación

A continuación se presentan las evidencias que acreditan la existencia de la vulnerabilidad:

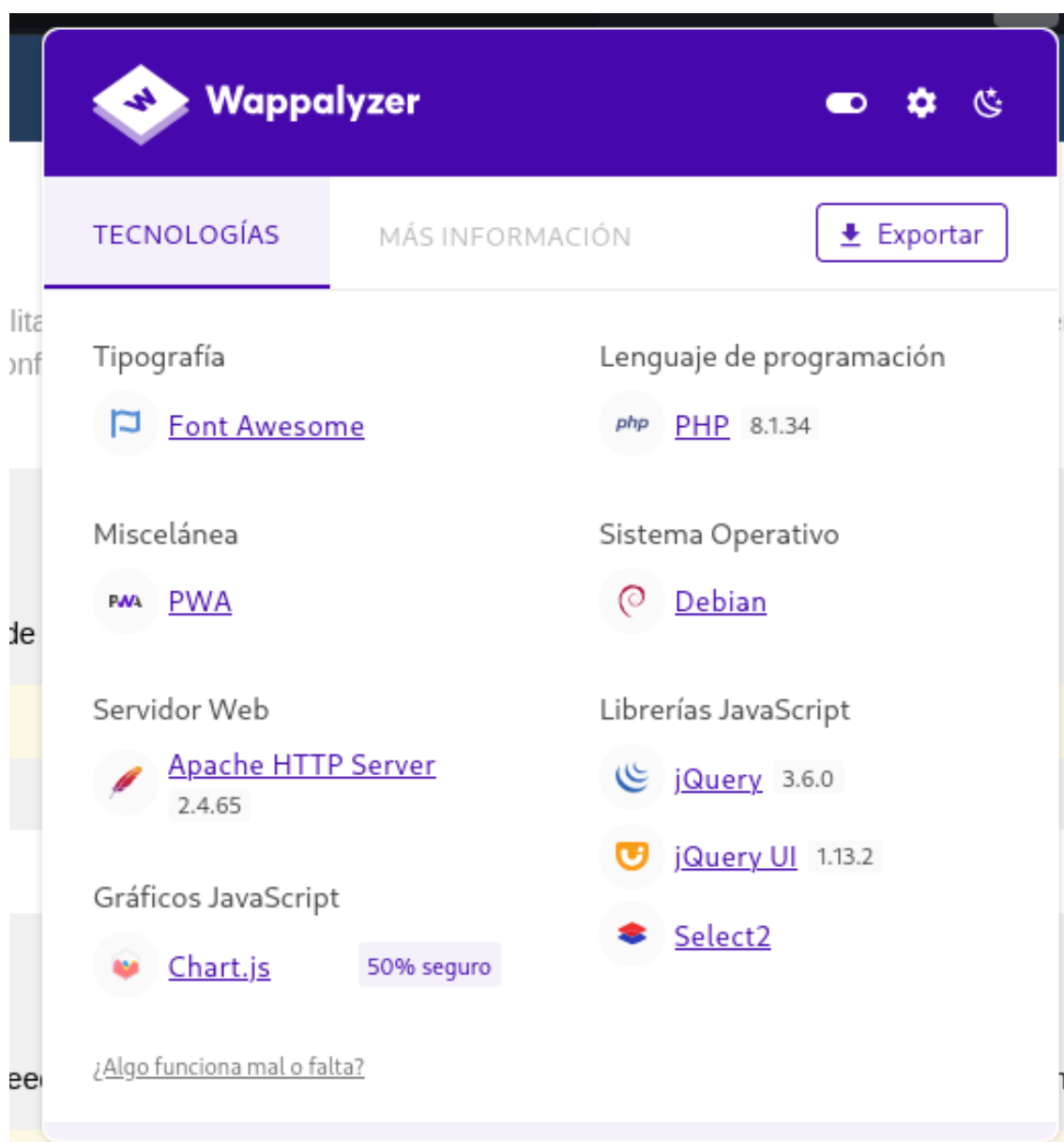


Figura 19: Piezas de software desactualizado

10. Divulgación de información sensible en mensajes de error

RSW-0826026001-10	Baja
Entorno afectado :	Dolibarr ERP/CRM 17.0.0
Funcionalidad :	Gestión de errores
Estado :	Pendiente
Tipo de vulnerabilidad :	Configuración
CVSS v3.1 :	3.7

Descripción

La directiva `display_errors` de PHP se encuentra activada en el entorno auditado. Como consecuencia, ante determinadas condiciones de error la aplicación devuelve al cliente avisos del intérprete que revelan información interna sobre el funcionamiento y la estructura del sistema.

Estos avisos exponen rutas internas del servidor y detalles de la lógica de la aplicación. Adicionalmente, ante una solicitud a un recurso inexistente, el servidor devuelve una página de error que revela la versión exacta del software empleado, proporcionando información útil para la fase de reconocimiento de un atacante.

Durante la revisión se han observado los siguientes aspectos:

- Los avisos de PHP revelan rutas internas del servidor (por ejemplo, `/var/www/html/...`).
- La página de error del servidor expone la versión de Apache (Apache/2.4.65 Debian).
- Directorios como `robots.txt` o `security.txt` muestran información extra.
- Esta información facilita la identificación de vectores de ataque conocidos.

Impacto derivado

La divulgación de rutas internas, versiones de software y detalles de la lógica de la aplicación proporciona a un atacante información valiosa durante la fase de reconocimiento. Este conocimiento facilita la identificación de vectores de ataque específicos y reduce el esfuerzo necesario para preparar un intento de compromiso.

Aunque esta divulgación no constituye por sí misma una vía de acceso directa, incrementa la eficacia de otros ataques al permitir al atacante adaptar sus técnicas a la configuración concreta del sistema. La exposición de versiones exactas facilita, en particular, la búsqueda de vulnerabilidades públicas aplicables al entorno.

Recomendación

Se recomienda desactivar la directiva `display_errors` en el entorno de producción, de modo que los errores se registren únicamente en ficheros de log internos y no se muestren al cliente. Esta medida evita la exposición de rutas internas y de detalles de la lógica de la aplicación ante usuarios no autorizados.

Explotación

A continuación se presentan las evidencias que acreditan la existencia de la vulnerabilidad:

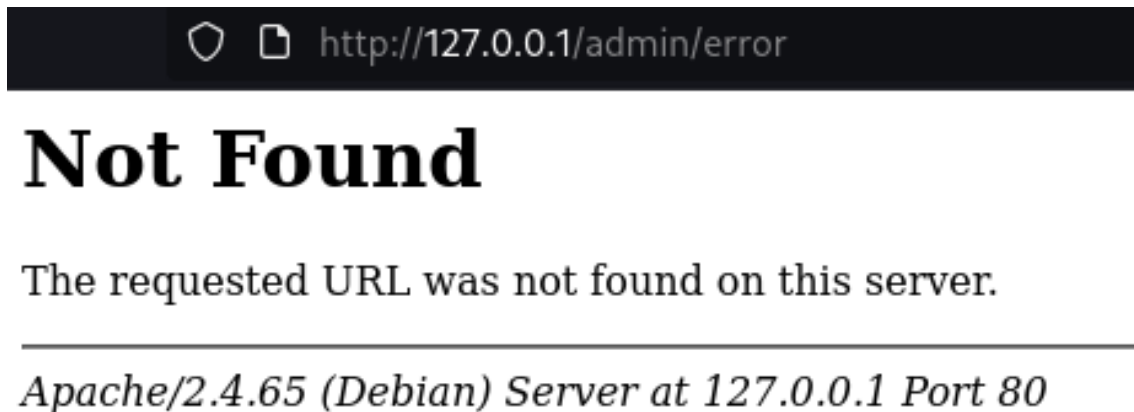


Figura 20: Error revela información

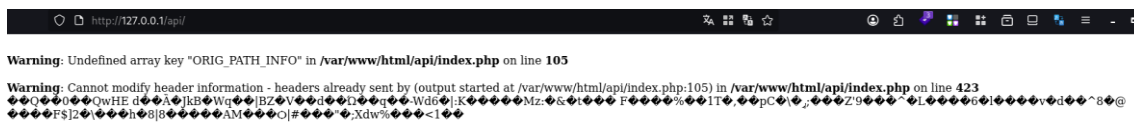


Figura 21: Error PHP

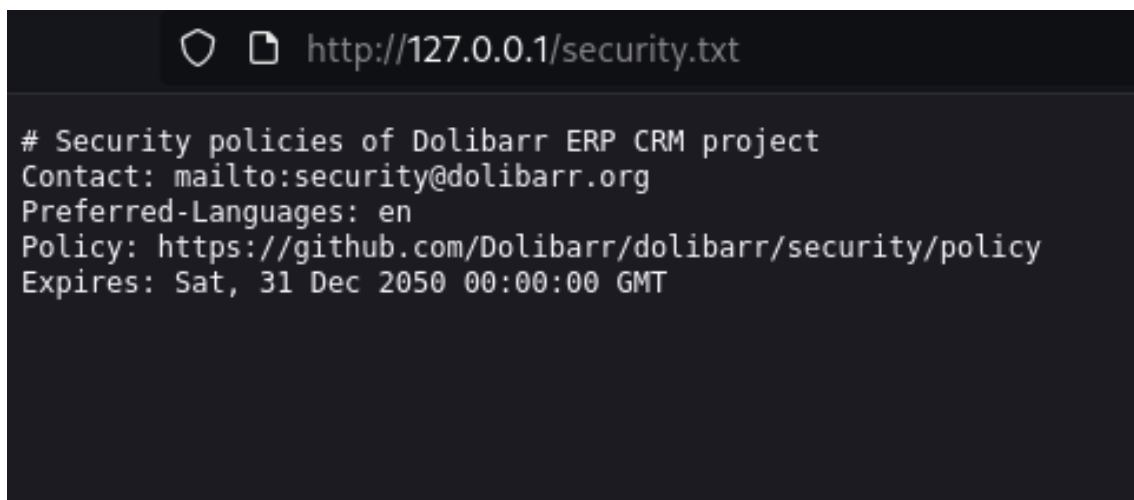


Figura 22: Security.txt

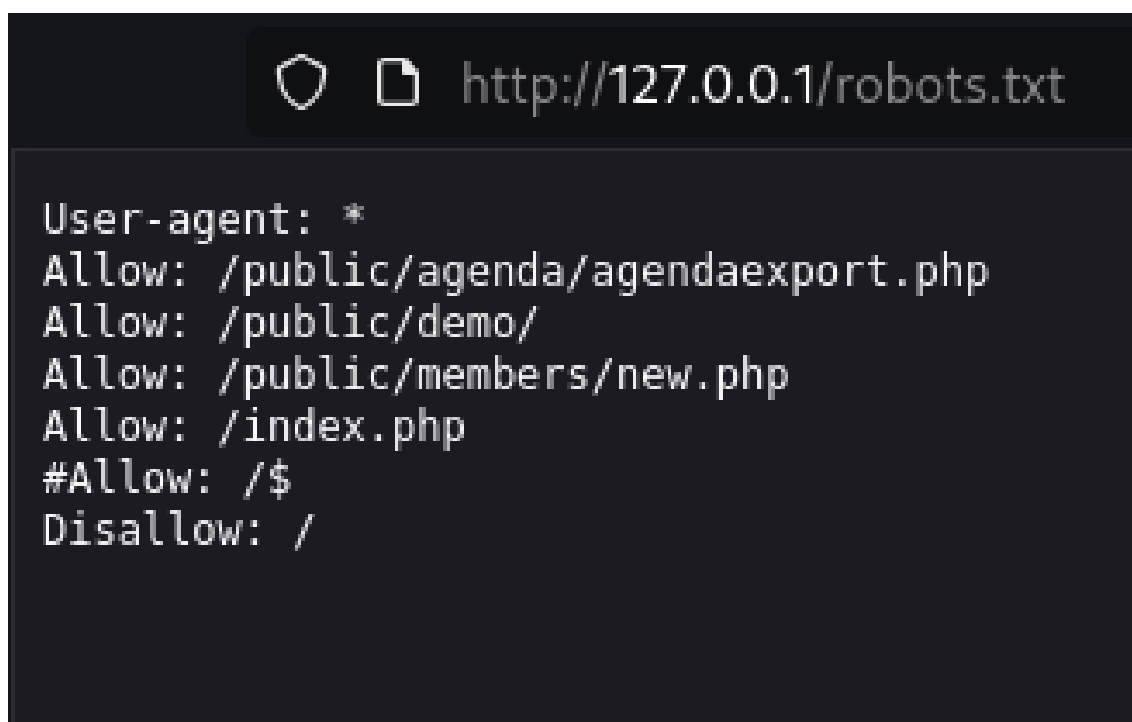


Figura 23: Robots.txt

3. Conclusiones

La auditoría de Dolibarr 17.0.0 ha confirmado que se trata de una versión con vulnerabilidades de severidad crítica y alta, principalmente relacionadas con la ejecución de código y la validación de entradas. Los dos vectores de ejecución remota de código y el XSS almacenado a través de la API tienen su corrección disponible a partir de la versión 17.0.1, lo que subraya la importancia de mantener el software actualizado.

Un aspecto especialmente relevante desde el punto de vista del análisis es el contraste entre los distintos controles de la aplicación: mientras que las defensas frente a la ejecución de código presentan fallos graves (filtros sensibles a mayúsculas, listas negras evadibles), los controles de acceso a ficheros y a objetos resisten adecuadamente los intentos de explotación. Esta divergencia evidencia que la seguridad de una aplicación no es homogénea y que un enfoque basado en listas negras es intrínsecamente más frágil que uno basado en listas blancas y codificación de salida.

Dolibarr 17.0.0 no es apta para su uso en producción sin actualizar. La aplicación de las recomendaciones recogidas en este informe, empezando por la actualización a una versión parcheada, el cifrado de credenciales y la habilitación de HTTPS, reduciría de forma sustancial la superficie de ataque y elevaría el nivel de seguridad del despliegue.

4. Criterios de clasificación

La clasificación de la severidad de cada vulnerabilidad se ha realizado empleando el estándar CVSS v3.1 (Common Vulnerability Scoring System), que proporciona una puntuación numérica de 0 a 10 a partir de un vector de métricas. Dicha puntuación se traduce en una categoría de criticidad, representada visualmente mediante un código de color a lo largo del informe:

Criticidad	Rango CVSS	Descripción
Crítica	9.0 – 10.0	Vulnerabilidad de máxima gravedad. Compromete el sistema de forma total. Requiere corrección inmediata.
Alta	7.0 – 8.9	Impacto trascendente sobre la seguridad. Requiere ser parcheada con la mayor brevedad posible.
Media	4.0 – 6.9	Impacto considerable, con alcance o facilidad de explotación moderados. Debe solucionarse con relativa rapidez.
Baja	0.1 – 3.9	Riesgo reducido. Impacto y/o facilidad de explotación bajos. Debe solucionarse en algún momento.
Informativa	0.0	No supone un riesgo por sí misma, pero puede facilitar otros ataques. Se aconseja aplicar contramedidas.

Cada vulnerabilidad se clasifica además según su tipología: Autorización, Autenticación, Cifrado, Configuración, Diseño, Mantenimiento, Sesión y Validación.

Estados de una vulnerabilidad

- **Pendiente:** la vulnerabilidad se comunica por primera vez y no consta que haya sido solucionada ni mitigada.
- **Solucionada:** reportada y corregida con contramedidas suficientes para anular su impacto.
- **Mitigada:** reportada y con contramedidas parciales que reducen su impacto, sin eliminarlo por completo.

5. Bitácora de verificación OWASP ASVS 5.0

Esta sección recoge la bitácora completa de verificación de controles del estándar OWASP ASVS 5.0 (Nivel 1) aplicada sobre la instancia auditada, con el resultado de cada control comprobado (Correcto, Deficiente o No aplica). Constituye el cierre del presente informe de auditoría.

Capítulo, sección y requisito	Descripción	Estado
V1:Codificación y Saneamiento V1.2:Prevención de Inyección V1.2.1	Verificar que la codificación de salida para una respuesta HTTP, un documento HTML o un documento XML es apropiada para el contexto requerido, como codificar los caracteres relevantes para elementos HTML, atributos HTML, comentarios HTML, CSS o campos de cabecera HTTP, para evitar modificar la estructura del mensaje o del documento.	Deficiente
V1:Codificación y Saneamiento V1.2:Prevención de Inyección V1.2.2	Verificar que cuando se construyen URLs dinámicamente, los datos no fiables se codifican según su contexto (por ejemplo, codificación URL o codificación base64url para parámetros de consulta o de ruta). Asegurar que solo se permiten protocolos URL seguros (por ejemplo, prohibir javascript: o data:).	No aplica
V1:Codificación y Saneamiento V1.2:Prevención de Inyección V1.2.3	Verificar que se utiliza codificación de salida o escapado al construir dinámicamente contenido JavaScript (incluido JSON), para evitar modificar la estructura del mensaje o del documento (y evitar inyección JavaScript y JSON).	No aplica
V1:Codificación y Saneamiento V1.2:Prevención de Inyección V1.2.4	Verificar que la selección de datos o las consultas a bases de datos (por ejemplo, SQL, HQL, NoSQL, Cypher) utilizan consultas parametrizadas, ORM, frameworks de entidades, o están protegidas de otro modo contra inyección SQL y otros ataques de inyección a bases de datos. Esto también es relevante al escribir procedimientos almacenados.	Correcto
V1:Codificación y Saneamiento V1.2:Prevención de Inyección V1.2.5	Verificar que la aplicación protege contra la inyección de comandos del sistema operativo y que las llamadas al SO utilizan consultas parametrizadas o codificación contextual de la salida de línea de comandos.	Deficiente
V1:Codificación y Saneamiento V1.3:Saneamiento V1.3.1	Verificar que toda entrada HTML no fiable procedente de editores WYSIWYG o similares es saneada usando una librería o funcionalidad del framework de saneamiento HTML reconocida y segura.	Deficiente
V1:Codificación y Saneamiento V1.3:Saneamiento V1.3.2	Verificar que la aplicación evita el uso de eval() u otras funcionalidades de ejecución dinámica de código, como Spring Expression Language (SpEL). Cuando no haya alternativa, cualquier entrada de usuario que se incluya debe ser saneada antes de su ejecución.	Deficiente
V1:Codificación y Saneamiento V1.5:Deserialización Segura V1.5.1	Verificar que la aplicación configura los analizadores XML para usar una configuración restrictiva y que las funcionalidades inseguras, como la resolución de entidades externas, están deshabilitadas para prevenir ataques de Entidad Externa XML (XXE).	No aplica
V2:Validación y Lógica de Negocio V2.1:Documentación de Validación y Lógica de Negocio V2.1.1	Verificar que la documentación de la aplicación define reglas de validación de entrada sobre cómo comprobar la validez de los datos frente a una estructura esperada. Esto puede incluir formatos de datos comunes como números de tarjeta de crédito, direcciones de correo electrónico, números de teléfono, o formatos de datos internos.	Deficiente
V2:Validación y Lógica de Negocio V2.2:Validación de Entrada V2.2.1	Verificar que la entrada es validada para cumplir las expectativas de negocio o funcionales. Esto debe realizarse mediante validación positiva contra una lista de valores, patrones y rangos permitidos, o comparando la entrada con una estructura esperada y límites lógicos según reglas predefinidas. Para L1, esto puede centrarse en entradas usadas para tomar decisiones de negocio o seguridad específicas. Para L2 y superior, debe aplicarse a todas las entradas.	Deficiente
V2:Validación y Lógica de Negocio V2.2:Validación de Entrada V2.2.2	Verificar que la aplicación está diseñada para aplicar la validación de entrada en una capa de servicio de confianza. Aunque la validación en el cliente mejora la usabilidad y debe fomentarse, no debe confiarse en ella como control de seguridad.	Deficiente
V2:Validación y Lógica de Negocio V2.3:Seguridad de la Lógica de Negocio V2.3.1	Verificar que la aplicación solo procesará los flujos de lógica de negocio para el mismo usuario en el orden de pasos esperado y sin saltarse pasos.	No aplica
V3:Seguridad del Frontend Web V3.2:Interpretación No Intencionada de Contenido V3.2.1	Verificar que existen controles de seguridad para evitar que los navegadores representen contenido o funcionalidad en respuestas HTTP en un contexto incorrecto (por ejemplo, cuando se solicita directamente una API, un fichero subido por un usuario u otro recurso). Los posibles controles podrían incluir: no servir el contenido a menos que los campos de cabecera de la petición HTTP (como Sec-Fetch-*) indiquen el contexto correcto, usar la directiva sandbox de la cabecera Content-Security-Policy o usar el tipo de disposición attachment en la cabecera Content-Disposition.	Correcto
V3:Seguridad del Frontend Web V3.2:Interpretación No Intencionada de Contenido V3.2.2	Verificar que el contenido destinado a mostrarse como texto, en lugar de renderizarse como HTML, se gestiona usando funciones de renderizado seguras (como createTextNode o textContent) para prevenir la ejecución no intencionada de contenido como HTML o JavaScript.	Correcto
V3:Seguridad del Frontend Web V3.3:Configuración de Cookies V3.3.1	Verificar que las cookies tienen el atributo 'Secure' establecido y, si no se usa el prefijo '__Host-' en el nombre de la cookie, debe usarse el prefijo '__Secure-'.	Deficiente
V3:Seguridad del Frontend Web V3.4:Cabeceras de Mecanismos de Seguridad del Navegador V3.4.1	Verificar que se incluye un campo de cabecera Strict-Transport-Security en todas las respuestas para aplicar una política de HTTP Strict Transport Security (HSTS). Debe definirse una edad máxima de al menos 1 año y, para L2 y superior, la política debe aplicarse también a todos los subdominios.	Deficiente
V3:Seguridad del Frontend Web V3.4:Cabeceras de Mecanismos de Seguridad del Navegador V3.4.2	Verificar que el campo de cabecera Access-Control-Allow-Origin del Cross-Origin Resource Sharing (CORS) es un valor fijo establecido por la	No aplica

Capítulo, sección y requisito	Descripción	Estado
	aplicación, o si se utiliza el valor del campo de cabecera de solicitud HTTP Origin, se valida contra una lista de orígenes de confianza. Cuando sea necesario usar 'Access-Control-Allow-Origin: *', verificar que la respuesta no incluye información sensible.	
V3:Seguridad del Frontend Web V3.5:Separación de Origen del Navegador V3.5.1	Verificar que, si la aplicación no se apoya en el mecanismo de preflight CORS para impedir solicitudes entre orígenes no permitidas que usen funcionalidad sensible, estas solicitudes se validan para garantizar que se originan desde la propia aplicación. Esto puede hacerse usando y validando tokens anti-falsificación o requiriendo campos de cabecera HTTP adicionales que no sean campos de cabecera seguros de CORS. Esto es para defenderse contra ataques de falsificación de solicitudes del navegador, comúnmente conocidos como falsificación de solicitudes entre sitios (CSRF).	Correcto
V3:Seguridad del Frontend Web V3.5:Separación de Origen del Navegador V3.5.2	Verificar que, si la aplicación se apoya en el mecanismo de preflight CORS para impedir el uso entre orígenes no permitidos de funcionalidad sensible, no es posible llamar a dicha funcionalidad con una solicitud que no desencadene una solicitud de preflight CORS. Esto puede requerir comprobar los valores de los campos de cabecera de solicitud 'Origin' y 'Content-Type' o usar un campo de cabecera adicional que no sea un campo de cabecera seguro de CORS.	No aplica
V3:Seguridad del Frontend Web V3.5:Separación de Origen del Navegador V3.5.3	Verificar que las solicitudes HTTP a funcionalidades sensibles usan métodos HTTP apropiados como POST, PUT, PATCH o DELETE, y no métodos definidos por la especificación HTTP como "seguros" como HEAD, OPTIONS o GET. Alternativamente, puede usarse la validación estricta de los campos de cabecera de solicitud Sec-Fetch-* para garantizar que la solicitud no se originó desde una llamada entre orígenes inapropiada, una solicitud de navegación o una carga de recursos (como una fuente de imagen) donde esto no sea esperado.	Correcto
V4:API y Servicios Web V4.1:Seguridad Genérica de Servicios Web V4.1.1	Verificar que cada respuesta HTTP con cuerpo de mensaje contiene un campo de cabecera Content-Type que coincide con el contenido real de la respuesta, incluyendo el parámetro charset para especificar la codificación de caracteres segura (por ejemplo, UTF-8, ISO-8859-1) según los Tipos de Medios de IANA, como "text/", "+xml" y "xml".	No aplica
V4:API y Servicios Web V4.4:WebSocket V4.4.1	Verificar que se usa WebSocket sobre TLS (WSS) para todas las conexiones WebSocket.	No aplica
V5:Gestión de Ficheros V5.2:Carga de Ficheros y Contenido V5.2.1	Verificar que la aplicación solo aceptará ficheros de un tamaño que pueda procesar sin causar pérdida de rendimiento o un ataque de denegación de servicio.	No aplica
V5:Gestión de Ficheros V5.2:Carga de Ficheros y Contenido V5.2.2	Verificar que cuando la aplicación acepta un fichero, ya sea por sí solo o dentro de un archivo comprimido como un zip, comprueba si la extensión del fichero coincide con una extensión esperada y valida que el contenido corresponde al tipo representado por la extensión. Esto incluye, entre otras cosas, la comprobación de los 'bytes mágicos' iniciales, la reescritura de imágenes y el uso de bibliotecas especializadas para la validación del contenido de ficheros. Para L1, esto puede centrarse únicamente en ficheros usados para tomar decisiones específicas de negocio o seguridad. Para L2 y superior, debe aplicarse a todos los ficheros aceptados.	Correcto
V5:Gestión de Ficheros V5.3:Almacenamiento de Ficheros V5.3.1	Verificar que los ficheros subidos o generados a partir de entradas no fiables y almacenados en una carpeta pública no se ejecutan como código de programa en el lado del servidor cuando se accede a ellos directamente mediante una solicitud HTTP.	Correcto
V5:Gestión de Ficheros V5.3:Almacenamiento de Ficheros V5.3.2	Verificar que cuando la aplicación crea rutas de fichero para operaciones con ficheros, en lugar de usar nombres de fichero enviados por el usuario, utiliza datos generados internamente o datos de confianza; o que si se deben usar nombres de fichero o metadatos de fichero enviados por el usuario, se aplica una validación y saneamiento estrictos. Esto es para proteger contra ataques de path traversal, inclusión local o remota de ficheros (LFI, RFI) y falsificación de solicitudes del lado del servidor (SSRF).	Correcto
V6:Autenticación V6.1:Documentación de Autenticación V6.1.1	Verificar que la documentación de la aplicación define cómo se utilizan controles como la limitación de velocidad, la anti-automatización y la respuesta adaptativa para defenderse de ataques como el relleno de credenciales y la fuerza bruta de contraseñas. La documentación debe dejar claro cómo se configuran estos controles y cómo previenen el bloqueo malicioso de cuentas.	Deficiente
V6:Autenticación V6.2:Seguridad de Contraseñas V6.2.1	Verificar que las contraseñas establecidas por los usuarios tengan al menos 8 caracteres de longitud, aunque se recomienda encarecidamente un mínimo de 15 caracteres.	Correcto
V6:Autenticación V6.2:Seguridad de Contraseñas V6.2.2	Verificar que los usuarios pueden cambiar su contraseña.	Correcto
V6:Autenticación V6.2:Seguridad de Contraseñas V6.2.3	Verificar que la funcionalidad de cambio de contraseña requiere la contraseña actual y la nueva del usuario.	No aplica
V6:Autenticación V6.2:Seguridad de Contraseñas V6.2.4	Verificar que las contraseñas enviadas durante el registro de cuenta o el cambio de contraseña se comprueban contra un conjunto disponible de, al menos, las 3000 contraseñas más comunes que coincidan con la política de contraseñas de la aplicación, por ejemplo, la longitud mínima.	Deficiente
V6:Autenticación V6.2:Seguridad de Contraseñas V6.2.5	Verificar que se pueden usar contraseñas de cualquier composición, sin reglas que limiten el tipo de caracteres permitidos. No debe exigirse un número mínimo de caracteres en mayúsculas, minúsculas, números o caracteres especiales.	Correcto
V6:Autenticación V6.2:Seguridad de Contraseñas V6.2.6	Verificar que los campos de entrada de contraseñas usan type=password para enmascarar la entrada. Las aplicaciones pueden permitir al usuario ver temporalmente toda la contraseña enmascarada o el último carácter escrito de la contraseña.	Correcto
V6:Autenticación V6.2:Seguridad de Contraseñas V6.2.7	Verificar que se permiten la funcionalidad de "pegar", los gestores de contraseñas del navegador y los gestores de contraseñas externos.	Correcto

Capítulo, sección y requisito	Descripción	Estado
V6:Autenticación V6.2:Seguridad de Contraseñas V6.2.8	Verificar que la aplicación verifica la contraseña del usuario exactamente como la recibe, sin ninguna modificación como truncamiento o transformación de mayúsculas/minúsculas.	Correcto
V6:Autenticación V6.3:Seguridad General de Autenticación V6.3.1	Verificar que los controles para prevenir ataques como el relleno de credenciales y la fuerza bruta de contraseñas se implementan según la documentación de seguridad de la aplicación.	Deficiente
V6:Autenticación V6.3:Seguridad General de Autenticación V6.3.2	Verificar que las cuentas de usuario predeterminadas (por ejemplo, "root", "admin" o "sa") no están presentes en la aplicación o están deshabilitadas.	Correcto
V6:Autenticación V6.4:Ciclo de Vida y Recuperación de Factores de Autenticación V6.4.1	Verificar que las contraseñas iniciales o códigos de activación generados por el sistema son generados de forma aleatoria y segura, siguen la política de contraseñas existente y caducan tras un corto periodo de tiempo o tras su uso inicial. Estos secretos iniciales no deben poder convertirse en la contraseña a largo plazo.	No aplica
V6:Autenticación V6.4:Ciclo de Vida y Recuperación de Factores de Autenticación V6.4.2	Verificar que no existen pistas de contraseña ni autenticación basada en conocimiento (las llamadas "preguntas secretas").	Correcto
V7:Gestión de Sesiones V7.2:Seguridad Fundamental de la Gestión de Sesiones V7.2.1	Verificar que la aplicación realiza toda la verificación de tokens de sesión usando un servicio backend de confianza.	No aplica
V7:Gestión de Sesiones V7.2:Seguridad Fundamental de la Gestión de Sesiones V7.2.2	Verificar que la aplicación utiliza tokens autocontenidos o de referencia generados dinámicamente para la gestión de sesiones, es decir, que no usa secretos API estáticos ni claves.	Correcto
V7:Gestión de Sesiones V7.2:Seguridad Fundamental de la Gestión de Sesiones V7.2.3	Verificar que si se usan tokens de referencia para representar sesiones de usuario, son únicos y generados usando un generador de números pseudoaleatorios criptográficamente seguro (CSPRNG) y poseen al menos 128 bits de entropía.	Correcto
V7:Gestión de Sesiones V7.2:Seguridad Fundamental de la Gestión de Sesiones V7.2.4	Verificar que la aplicación genera un nuevo token de sesión al autenticarse el usuario, incluida la re-autenticación, y termina el token de sesión actual.	Correcto
V7:Gestión de Sesiones V7.4:Terminación de Sesión V7.4.1	Verificar que cuando se activa la terminación de sesión (como el cierre de sesión o la expiración), la aplicación no permite ningún uso adicional de la sesión. Para tokens de referencia o sesiones con estado, esto significa invalidar los datos de la sesión en el backend de la aplicación. Las aplicaciones que usan tokens autocontenidos necesitarán una solución como mantener una lista de tokens terminados, rechazar tokens producidos antes de una fecha y hora por usuario, o rotar una clave de firma por usuario.	No aplica
V7:Gestión de Sesiones V7.4:Terminación de Sesión V7.4.2	Verificar que la aplicación termina todas las sesiones activas cuando una cuenta de usuario es deshabilitada o eliminada (como cuando un empleado deja la empresa).	No aplica
V8:Autorización V8.1:Documentación de Autorización V8.1.1	Verificar que la documentación de autorización define reglas para restringir el acceso a nivel de función y específico de datos basándose en los permisos del consumidor y los atributos del recurso.	No aplica
V8:Autorización V8.2:Diseño General de Autorización V8.2.1	Verificar que la aplicación garantiza que el acceso a nivel de función está restringido a los consumidores con permisos explícitos.	Correcto
V8:Autorización V8.2:Diseño General de Autorización V8.2.2	Verificar que la aplicación garantiza que el acceso específico a datos está restringido a los consumidores con permisos explícitos para elementos de datos específicos, para mitigar la referencia directa a objetos inseguros (IDOR) y la autorización rota a nivel de objeto (BOLA).	Correcto
V8:Autorización V8.3:Autorización a Nivel de Operación V8.3.1	Verificar que la aplicación aplica las reglas de autorización en una capa de servicio de confianza y no depende de controles que un consumidor no fiable pueda manipular, como el JavaScript del lado del cliente.	Correcto
V9:Tokens Autocontenidos V9.1:Origen e Integridad del Token V9.1.1	Verificar que los tokens autocontenidos son validados usando su firma digital o MAC para protegerse contra la manipulación antes de aceptar el contenido del token.	No aplica
V9:Tokens Autocontenidos V9.1:Origen e Integridad del Token V9.1.2	Verificar que solo los algoritmos de una lista de permitidos pueden usarse para crear y verificar tokens autocontenidos, para un contexto dado. La lista de permitidos debe incluir los algoritmos permitidos, idealmente solo algoritmos simétricos o asimétricos, y no debe incluir el algoritmo 'None'. Si se deben soportar tanto los simétricos como los asimétricos, se necesitarán controles adicionales para prevenir la confusión de claves.	No aplica
V9:Tokens Autocontenidos V9.1:Origen e Integridad del Token V9.1.3	Verificar que el material de clave utilizado para validar los tokens autocontenidos proviene de fuentes preconfiguradas de confianza para el emisor del token, evitando que los atacantes especifiquen fuentes y claves no fiables. Para JWTs y otras estructuras JWS, las cabeceras como 'jku', 'x5u' y 'jwk' deben validarse contra una lista de fuentes de confianza.	No aplica
V9:Tokens Autocontenidos V9.2:Contenido del Token V9.2.1	Verificar que, si un intervalo de validez está presente en los datos del token, el token y su contenido solo se aceptan si el tiempo de verificación está dentro de este intervalo de validez. Por ejemplo, para JWTs, las reclamaciones 'nbf' y 'exp' deben verificarse.	No aplica
V10:OAuth y OIDC V10.4:Servidor de Autorización OAuth V10.4.1	Verificar que el servidor de autorización valida las URIs de redirección basándose en una lista de URIs pre-registradas específicas del cliente usando comparación exacta de cadenas.	No aplica
V10:OAuth y OIDC V10.4:Servidor de Autorización OAuth V10.4.2	Verificar que, si el servidor de autorización devuelve el código de autorización en la respuesta de autorización, solo puede usarse una vez para una solicitud de token. Para la segunda solicitud válida con un código de autorización que ya se ha usado para emitir un token de acceso, el servidor de autorización debe rechazar la solicitud de token y revocar cualquier token emitido relacionado con el código de autorización.	No aplica
V10:OAuth y OIDC V10.4:Servidor de Autorización OAuth V10.4.3	Verificar que el código de autorización tiene una vida corta. El tiempo de vida máximo puede ser de hasta 10 minutos para aplicaciones L1 y L2 y de hasta 1 minuto para aplicaciones L3.	No aplica
V10:OAuth y OIDC V10.4:Servidor de Autorización OAuth V10.4.4	Verificar que para un cliente dado, el servidor de autorización solo permite el uso de los tipos de concesión que este cliente necesita usar. Obsérvese que los tipos de concesión 'token' (flujo implícito) y 'password' (flujo de	No aplica

Capítulo, sección y requisito	Descripción	Estado
	Credenciales de Contraseña del Propietario del Recurso) ya no deben usarse.	
V10:OAuth y OIDC V10.4:Servidor de Autorización OAuth V10.4.5	Verificar que el servidor de autorización mitiga los ataques de repetición de tokens de actualización para clientes públicos, preferiblemente usando tokens de actualización vinculados al emisor, es decir, Demostración de Posesión de Prueba (DPoP) o Tokens de Acceso Vinculados a Certificado usando TLS mutuo (mTLS). Para aplicaciones L1 y L2, puede usarse la rotación de tokens de actualización. Si se usa la rotación de tokens de actualización, el servidor de autorización debe invalidar el token de actualización tras su uso, y revocar todos los tokens de actualización para esa autorización si se proporciona un token de actualización ya usado e invalidado.	No aplica
V11:Criptografía V11.3:Algoritmos de Cifrado V11.3.1	Verificar que no se usan modos de bloque inseguros (por ejemplo, ECB) ni esquemas de relleno débiles (por ejemplo, PKCS#1 v1.5).	No aplica
V11:Criptografía V11.3:Algoritmos de Cifrado V11.3.2	Verificar que solo se usan cifrados y modos aprobados, como AES con GCM.	No aplica
V11:Criptografía V11.4:Funciones de Hash y Basadas en Hash V11.4.1	Verificar que solo se usan funciones hash aprobadas para casos de uso criptográfico general, incluyendo firmas digitales, HMAC, KDF y generación de bits aleatorios. Las funciones hash no permitidas, como MD5, no deben usarse para ningún propósito criptográfico.	No aplica
V12:Comunicación Segura V12.1:Orientación General de Seguridad TLS V12.1.1	Verificar que solo las versiones más recientes recomendadas del protocolo TLS están habilitadas, como TLS 1.2 y TLS 1.3. La versión más reciente del protocolo TLS debe ser la opción preferida.	Deficiente
V12:Comunicación Segura V12.2:Comunicación HTTPS con Servicios Expuestos Externamente V12.2.1	Verificar que se usa TLS para toda la conectividad entre un cliente y los servicios basados en HTTP expuestos externamente, y que no retrocede a comunicaciones inseguras o sin cifrar.	Deficiente
V12:Comunicación Segura V12.2:Comunicación HTTPS con Servicios Expuestos Externamente V12.2.2	Verificar que los servicios expuestos externamente usan certificados TLS públicamente de confianza.	Deficiente
V13:Configuración V13.4:Fuga de Información No Intencionada V13.4.1	Verificar que la aplicación se despliega sin metadatos de control de versiones, incluyendo las carpetas .git o .svn, o de forma que estas carpetas sean inaccesibles tanto externamente como para la propia aplicación.	Correcto
V14:Protección de Datos V14.2:Protección General de Datos V14.2.1	Verificar que los datos sensibles solo se envían al servidor en el cuerpo del mensaje HTTP o en los campos de cabecera, y que la URL y la cadena de consulta no contienen información sensible, como una clave API o un token de sesión.	Correcto
V14:Protección de Datos V14.3:Protección de Datos en el Cliente V14.3.1	Verificar que los datos autenticados se borran del almacenamiento del cliente, como el DOM del navegador, después de que el cliente o la sesión sean terminados. El campo de cabecera de respuesta HTTP 'Clear-Site-Data' puede ayudar con esto, pero el lado del cliente también debe ser capaz de limpiar si la conexión al servidor no está disponible cuando se termina la sesión.	No aplica
V15:Codificación Segura y Arquitectura V15.1:Documentación de Codificación Segura y Arquitectura V15.1.1	Verificar que la documentación de la aplicación define plazos de remediación basados en el riesgo para las versiones de componentes de terceros con vulnerabilidades y para la actualización de bibliotecas en general, para minimizar el riesgo derivado de estos componentes.	No aplica
V15:Codificación Segura y Arquitectura V15.2:Arquitectura de Seguridad y Dependencias V15.2.1	Verificar que la aplicación solo contiene componentes que no han superado los plazos documentados de actualización y remediación.	Deficiente
V15:Codificación Segura y Arquitectura V15.3:Codificación Defensiva V15.3.1	Verificar que la aplicación solo devuelve el subconjunto requerido de campos de un objeto de datos. Por ejemplo, no debe devolver un objeto de datos completo, ya que algunos campos individuales no deben ser accesibles para los usuarios.	Correcto